

Ziren WHIR Soundness

A Production-Sound Multilinear PCS Pipeline for STARK Recursion
with Late-Binding, Cross-Binding, and Jagged Packing

ProjectZKM Research¹

¹ProjectZKM

April 2026

Abstract

We present a production-sound integration of the WHIR multilinear polynomial commitment scheme into the Ziren zkVM’s STARK pipeline. Our work closes three soundness gaps in the existing WHIR-fast-path: (i) unbound transition constraints, fixed by a sumcheck-based zerocheck protocol; (ii) unbound lookup arguments, fixed by a sumcheck reduction of LogUp-GKR; and (iii) unbound per-row multilinear-extension openings, fixed by a novel *late-binding* WHIR adapter that allows evaluation points to be sampled *after* commitment, in service of the GKR-derived row coordinates that no upstream WHIR API supports natively.

We further introduce a *cross-binding* protocol (a Brakedown- style spot-check argument) that ties the FRI commit and the WHIR late-binding commit to the same trace, eliminating a divergent-trace attack that the dual-commit shipping iteration leaves open. An independent contribution is the *jagged late-binding* construction: combining variable-height chip-trace packing with the late-binding adapter via a sumcheck reduction, reducing per-shard proof size by a factor of 57–85× (29.5 MB → 488 KB on a fibonacci shard) at 1.3–2.1× prover-compute cost.

All host-side prover and verifier code is implemented and tested in the open-source Ziren codebase. Cryptographic helpers ship with end-to-end round-trip and tamper-rejection unit tests; soundness mechanisms are validated by our 62/62 relevant unit tests (24 in the late-binding modules, 18 in the recursion-circuit WHIR verifier, 9 in zerocheck and LogUp-GKR, plus the existing suite). We also profile the construction at scale on the fibonacci, JSON, and keccak workloads, document the workload-dependent compute trade-off, and identify upstream parallelisation of `p3-whir`’s sumcheck rounds as the critical next-step optimisation.

Keywords: STARKs · multilinear polynomial commitment · WHIR · soundness · recursion · zero-knowledge virtual machines · Plonky3.

Contents

1	Introduction	4
1.1	Contributions	4
1.2	Implementation and Open-Source Release	5
1.3	Paper Outline	5
2	Preliminaries	5
2.1	Notation	5
2.2	STARKs and the Ziren Pipeline	5
2.3	Sumcheck Protocol	6
2.4	WHIR	6
2.5	LogUp-GKR	6

2.6	Plonky3 Architecture	6
3	Background and Motivation	7
3.1	The starting point	7
3.2	The Plonky3 / WHIR upgrade	7
3.3	The architecture of this work	7
3.4	Status of the work	9
4	Plonky3 Upgrade	10
4.1	Per-round PoW witnesses	10
4.2	Log-arity observations and opened-values restoration	10
4.3	<code>check_witness</code> early-return for zero PoW bits	10
4.4	<code>is_div_active</code> preprocessed column	10
4.5	Result	10
5	Soundness Fix Design: Making WHIR Mode Production-Sound	10
5.1	Current State	10
5.2	What This Leaves Unchecked	11
5.3	Target Soundness Design	11
5.4	Chosen Design: Option B	12
5.5	Projected Performance Impact	13
5.6	Immediate Action Items	14
6	System Design: Proof-System Architecture	14
6.1	Proof-System Overview	14
6.2	Data Layout: Jagged Polynomial Commitment	14
6.3	Polynomial Commitment: WHIR	15
6.4	Late-Binding WHIR	15
6.5	Jagged + Late-Binding Reduction	16
6.6	Cross-Binding	17
6.7	Soundness Layers Summary	18
6.8	Implementation Activation Matrix	18
7	Phase 2a: Zerocheck Integration	18
7.1	What Ships	18
7.2	End-to-End Test Results	19
7.3	Phase 2a Scope Limitation (By Design)	20
7.4	Files Changed (Phase 2a)	20
8	Phase 2b: LogUp-GKR Protocol Core	20
8.1	Fraction-Sum Combine	20
8.2	GKR Reduction	21
8.3	Soundness Tests	21
8.4	Public API	21
8.5	Integration Path (Phase 2b Steps 2 – 4)	22
8.6	Combined Soundness Picture	22
9	LogUp-GKR: Bug, Fix, and End-to-End Integration	23
9.1	The Soundness Bug (And Why It Was Hard to Spot)	23
9.2	The Correct Reduction	23
9.3	Implementation	24
9.4	Closing the Soundness Loop	25
9.5	Files Touched	25

10 Phase 2c: WHIR Late-Binding and Per-Chip Row-MLE	25
10.1 The Problem with the Standard WHIR Adapter	26
10.2 Late-Binding Adapter	26
10.3 Architectural Choice for Multi-Column Binding	27
10.4 Restructured Leaf-Claim Reconstruction	27
10.4.1 The Fix: Fingerprint Re-Batching at GKR Time	28
10.5 The VEIL Multilinear PCS Construction (the actual answer)	28
10.6 Cross-Binding Plan (Closing the First-Iteration Gap)	30
10.7 Implementation Plan	32
11 Performance Evaluation: Plonky3 Upgrade and WHIR PCS	33
11.1 Test Environment	33
11.2 Core Proof Performance	34
11.3 Compressed Proof Performance	34
11.4 Proof Size Comparison	34
11.5 Key Findings	34
11.6 Soundness Caveat for the Current WHIR Fast Path	35
11.7 Wrap / Groth16 Status	35
11.8 Known Failures (Pre-existing, Not WHIR-related)	35
11.9 Notes	35
12 Related Work	36
12.1 WHIR PCS Constructions	36
12.2 Sumcheck-Based STARK Verification	36
12.3 SP1 Hypercube and Recursion Architectures	36
12.4 Brakedown and Spot-Check Protocols	36
12.5 Jagged Polynomial Commitments	37
13 Conclusion and Future Work	37
13.1 Future Work	37
13.2 Acknowledgements	38
A Implementation Status and Next Steps	38
A.1 What ships today	38
A.2 What's next	39
A.3 Soundness summary	39
B Changelog: Phase 2c Late-Binding Implementation	39
B.1 Shipped (2026-04-16 / 17)	39
B.2 Cross-Binding (FRI $\leftrightarrow$ WHIR consistency)	41
B.3 Plonky3 <code>MerkleTree::Clone</code> Fix (Pre-Existing Bug Found)	41
B.4 Recursion Circuit Host-Shape Verifier Benchmark	42
B.5 Recursion Circuit: Full DSL $\rightarrow$ AIR Pipeline for SumcheckVerify	43
B.6 Recursion-Circuit WHIR Verifier Scaffolding (2026-04-17)	44
B.7 Final Soundness Chain Status (2026-04-17, end of session)	45
B.8 Phase 2c+ Session Summary (2026-04-17)	45
B.9 Phase 2c+ Jagged + Late-Binding (Shipped 2026-04-17)	46
B.10 Phase 2c+ Performance Scoreboard	50
B.11 Cost Optimisation: VEIL Lemma 4.7	54
B.12 Crisp Next Steps (Code Pointers)	54

1 Introduction

Zero-knowledge virtual machines (zkVMs) compile general-purpose computation into succinct cryptographic proofs. The Ziren zkVM [8] is a MIPS-based zkVM built on the Plonky3 [7] STARK toolkit, currently shipping with FRI as its polynomial commitment scheme (PCS). Although FRI offers excellent verification time, modern multilinear PCS such as WHIR [1] promise simultaneously smaller proofs, faster verification, and a more natural multilinear-extension (MLE) view of trace data — all of which are increasingly relevant as zkVMs adopt hypercube-style architectures [9].

This paper documents the cryptographic and engineering work required to migrate Ziren’s STARK pipeline from FRI to a production-sound WHIR-based pipeline. The transition is non-trivial: the existing “WHIR fast path” shipped in Ziren is a structural skip of permutation traces and quotient evaluation, with several unbound soundness gaps. We close all of them.

1.1 Contributions

1. **Sumcheck-based zerocheck for transition constraints (§7).** Replaces the quotient polynomial argument with a sumcheck protocol proving $\sum_{b \in \{0,1\}^m} \text{eq}(r, b) \cdot C(b) = 0$, with provable soundness $O(d \cdot m/|\mathbb{F}|)$ for constraint degree d .
2. **Sumcheck-based LogUp-GKR for lookup arguments (§9).** We identify and fix a critical soundness bug in the initial folding-only implementation: the combine identity $(N(z\|0) \cdot D(z\|1) + D(z\|0) \cdot N(z\|1), D(z\|0) \cdot D(z\|1))$ is degree-2 in each coordinate, which only matches the verifier’s degree-1 multilinear N_{k+1}, D_{k+1} at Boolean points. Our replacement protocol uses a per-layer sumcheck reduction over a degree-3 round polynomial to ensure soundness at random points.
3. **Late-binding WHIR adapter (§10).** The upstream WHIR `MultilinearPcs` adapter requires evaluation points *at commit time*, which is incompatible with our protocol where the per-chip row coordinates r_{row} are derived *after* commit via GKR. We construct an in-tree adapter built directly on `p3-whir` internals (`CommitmentWriter`, `Verifier`, `EqStatement`) that exposes single-column and multi-column commit + post-commit `add_late_claim` + open API surfaces. The cryptographic substrate is unchanged from upstream WHIR; only the transcript ordering differs.
4. **Cross-binding (C.1) Brakedown spot-check protocol.** In a dual-commit setting where FRI and WHIR both commit to the same trace, a malicious prover can commit divergent traces $A \neq B$. We close this with a Brakedown-style spot-check argument [5]: K random trace positions are sampled post-commit; the prover provides merkle paths against *both* commits at each position. Tampering on either side is detected with soundness $1 - (1 - 1/N)^K$. For $K = 80$ and $N = 2^{17}$ this gives $\sim 2^{-80}$ missed-detection probability.
5. **Jagged late-binding construction.** We combine Jagged [6] variable-height trace packing with the late-binding adapter via a sumcheck reduction that ties per-chip per-column row-MLE claims to a single dense commitment. This collapses ~ 570 per-chip-per-column WHIR commits to a single shard-level commit, reducing proof size by a factor of $57\text{--}85\times$ in our evaluation (§11).
6. **Recursion-circuit WHIR verifier scaffolding.** We port the host-side verification logic into the recursion-circuit through a series of focused sub-systems: a `SumcheckVerify` chip (full DSL $\rightarrow$ AIR pipeline, seven layers), a merkle-path verifier built atop the existing Poseidon2 chip, OOD evaluation primitives, and STIR query position derivation. Each sub-system is independently testable and serves as a stepping stone toward in-circuit `verify_whir_pcs`.

1.2 Implementation and Open-Source Release

All host-side prover and verifier code is implemented in the open-source Ziren codebase, opt-in via two environment variables: `ZIREN_USE_WHIR=1` (selects the WHIR fast path with zerocheck and LogUp-GKR), and `ZIREN_LATE_BINDING=jagged` (selects the jagged late-binding path). A third flag `ZIREN_CROSS_BINDING=1` activates the cross-binding spot-check protocol. Defaults preserve the original FRI pipeline so existing deployments are unaffected.

We test each cryptographic mechanism via end-to-end round-trip and tamper-rejection unit tests, plus end-to-end smoke tests on the fibonacci, JSON, and keccak workloads. At the time of writing, 24 late-binding-related tests, 9 zerocheck and LogUp-GKR tests, 18 recursion-circuit WHIR verifier tests, and the existing 34 recursion-core tests all pass. Compressed-proof verification of each smoke-tested workload completes successfully.

1.3 Paper Outline

§2 reviews the necessary preliminaries (STARK, sumcheck, WHIR, LogUp-GKR, Plonky3 architecture). §3 sets the engineering context, summarises the Plonky3 upgrade work, and tabulates implementation status. §4 details the recursion-circuit FRI verifier upgrade required by upstream Plonky3 protocol changes. §5 formalises the threat model of the existing WHIR fast path. §§7–10 present the three new constructions: zerocheck (Phase 2a), LogUp-GKR (Phase 2b), and late-binding WHIR with cross-binding and jagged packing (Phase 2c+). §11 reports performance measurements. §12 positions the work against prior constructions. §13 concludes with future directions. Appendices document detailed implementation status and a session-by-session changelog.

2 Preliminaries

2.1 Notation

We work over a finite field $\mathbb{F}$ (KoalaBear in our deployment, $|\mathbb{F}| \approx 2^{31}$) and an extension $\mathbb{F}_q$ with $q = |\mathbb{F}|^D$ for $D = 4$, giving challenger soundness $\sim 2^{-124}$. $N = 2^n$ denotes a power-of-two domain size, typically the height of a chip’s trace. Bold lowercase letters ($\mathbf{x}, \mathbf{r}, \mathbf{z}$) denote vectors of field elements; uppercase (P, Q, F) denote polynomials.

For a vector of evaluations $\mathbf{v} \in \mathbb{F}^{2^m}$, the *multilinear extension* (MLE) $\tilde{\mathbf{v}} : \mathbb{F}^m \rightarrow \mathbb{F}$ is the unique multilinear polynomial satisfying $\tilde{\mathbf{v}}(\mathbf{b}) = v_{\text{idx}(\mathbf{b})}$ for $\mathbf{b} \in \{0, 1\}^m$. We use the multilinear equality polynomial $\text{eq}(\mathbf{r}, \mathbf{x}) = \prod_{i=1}^m (r_i x_i + (1 - r_i)(1 - x_i))$, which satisfies $\tilde{\mathbf{v}}(\mathbf{r}) = \sum_{\mathbf{b} \in \{0, 1\}^m} \text{eq}(\mathbf{r}, \mathbf{b}) \cdot v_{\text{idx}(\mathbf{b})}$.

2.2 STARKs and the Ziren Pipeline

A STARK proves correct execution of a virtual-machine program by: (1) expressing the program’s per-cycle behaviour as an algebraic intermediate representation (AIR), (2) committing to the resulting trace via a polynomial commitment scheme (PCS), (3) verifying that the committed trace satisfies the AIR’s transition and boundary constraints (via a quotient polynomial argument or a sumcheck), and (4) verifying lookups (via the LogUp [3] family of arguments).

Ziren’s main branch implements step (2) via FRI on a multiplicative subgroup, with separate commits to main, permutation, and quotient traces. Step (3) is the standard quotient identity $Q(\zeta) \cdot Z_H(\zeta) = C_{\text{batched}}(\zeta)$ at a random univariate point ζ .

2.3 Sumcheck Protocol

The sumcheck protocol [4] reduces a claim about a polynomial sum over the Boolean hypercube to a single polynomial evaluation:

$$\text{Claim: } \sum_{\mathbf{b} \in \{0,1\}^m} F(\mathbf{b}) = v.$$

For each round $i \in [m]$, the prover sends a univariate polynomial $p_i(X) = \sum_{\mathbf{b}' \in \{0,1\}^{m-i}} F(r_1, \dots, r_{i-1}, X, \mathbf{b}')$ of degree $\deg(F)$, and the verifier checks $p_i(0) + p_i(1) = s_i$ (where $s_1 = v$ and $s_{i+1} = p_i(r_i)$ for verifier-sampled $r_i \in \mathbb{F}_q$). After m rounds, the verifier checks $F(r_1, \dots, r_m) = s_{m+1}$. Soundness error is $d \cdot m / |\mathbb{F}_q|$ for $\deg(F) = d$.

2.4 WHIR

WHIR [1] is an interactive oracle proof of proximity (IOPP) for Reed–Solomon (RS) codes that supports multilinear polynomial commitments with $O(\log n)$ verifier work and proof sizes 5–10× smaller than FRI at equivalent security.

A WHIR commitment to $f \in \mathbb{F}^{2^n}$ encodes f ’s coefficients into an RS codeword on a coset of size 2^{n+r} ($r = \log\text{-inv-rate}$) and merkle-commits the row hashes. The open phase combines sumcheck folding rounds with STIR query openings.

The Plonky3 implementation `p3-whir` provides a `MultilinearPcs` adapter requiring opening points *at commit time* (absorbed into Fiat-Shamir before the batching challenge α is sampled). This is the standard soundness ordering when opening points are pre-known; our work (§10) constructs an alternative *late-binding* ordering for protocols where the opening point is derived post-commit.

2.5 LogUp-GKR

LogUp [3] reduces a multiset-equality lookup argument to a sum-of-fractions identity

$$\sum_{i \in \text{senders}} \frac{m_i}{\alpha - f_i} = \sum_{j \in \text{receivers}} \frac{m_j}{\alpha - f_j},$$

where m_* are multiplicities and f_* are interaction fingerprints. GKR [3] verifies this fraction-sum identity via a layered tree of fraction combinations $(a, b) \oplus (c, d) = (ad + bc, bd)$, opening only the root (num, denom) and verifying num = 0.

The verification reduces tree-root claims to leaf claims via sumcheck rounds at each layer. The leaf claims are MLE evaluations of multiplicities and fingerprints at a sumcheck-derived point r_{row} , which is therefore *post-commit*. This post-commit derivation is the central structural difficulty for binding the leaf claims to a WHIR commitment of the trace.

2.6 Plonky3 Architecture

Ziren is built on Plonky3 [7], which provides chip-based AIR composition, Fiat-Shamir challenger, MMCS (mixed-matrix commitment scheme) merkle trees, and a recursion compiler. Our work modifies neither the chip composition nor the recursion compiler core; we add new chips (sumcheck verification), new instructions (sumcheck verify), new prover/verifier paths (WHIR fast path), and new helpers (late-binding, cross-binding, jagged + late-binding) on top of the existing infrastructure.

3 Background and Motivation

3.1 The starting point

Ziren’s main branch ships a STARK prover with FRI as the polynomial commitment scheme. Each shard commits the main trace, the permutation trace (for LogUp lookups), and a quotient polynomial (for transition constraints), then opens all three at a univariate point ζ . The verifier replays the same three opens and checks the AIR identities: constraint-quotient relationship, lookup accumulation, cumulative-sum closure.

This is the standard univariate-FRI STARK pipeline. It has the disadvantages typical of FRI: large per-shard proofs (megabytes for RISC-V workloads), expensive recursion (many merkle paths per query), and a large recursion circuit (the FRI verifier must be expressed in the recursion AIR, with all the per-round PoW witnesses, log-arity observations, and merkle-path checks).

3.2 The Plonky3 / WHIR upgrade

Two upstream developments motivated a major upgrade:

1. **Plonky3 protocol updates.** Upstream Plonky3 added per-round PoW witnesses, log-arity observations, and a restructured opened-values absorption pattern to the FRI verifier. This is necessary for proper Fiat-Shamir binding with adaptive folding factors but breaks compatibility with any recursion circuit pinned to the older protocol.
2. **WHIR multilinear PCS.** The WHIR protocol [1] reduces multilinear polynomial commitments to a sequence of sumcheck rounds with folding, achieving $O(\log n)$ verifier work and proof sizes 5–10 $\times$ smaller than FRI for the same security level. Plonky3’s `p3-whir` adapter exposes WHIR as a `MultilinearPcs`.

3.3 The architecture of this work

The work documented in this whitepaper does three things, in order:

1. **Plonky3 upgrade.** Bring Ziren’s recursion circuit and FRI verifier in line with the new Plonky3 protocol. Includes per-round PoW witnesses in the recursion FRI verifier, fixed `check_witness` for empty PoW, and a `is_div_active` preprocessed column to gate `DivF/DivE` in the recursion AIR (see §4).
2. **Phase 2 soundness fix.** Replace the quotient and permutation-trace mechanisms with sumcheck-based equivalents that work multilinearly: zerocheck (Phase 2a) and LogUp-GKR (Phase 2b). This is what makes the WHIR fast path actually sound, not just a structural skip (see §7, §9).
3. **Late-binding WHIR adapter (Phase 2c).** The remaining soundness gap after Phase 2b: per-chip row-MLE openings (`LogUpRowOpening`) are sent in the proof but not cryptographically tied to the main-trace commitment. Closing this gap requires opening the WHIR commitment at multilinear points sampled *after* the commit, which the upstream adapter’s API forbids. This work introduces a late-binding adapter that correctly handles post-commit point selection (see §10).

3.4 Status of the work

Item	Status	Tests
Plonky3 upgrade (recursion FRI verifier)	shipped, compressed proofs verify	—
Cherry-pick aggregation fix (7c2071e)	shipped on feat/upgrade-plonky3	—
Aggregation example with WHIR (ZK stages)	all stages suc- ceed; gnark wrap blocked on stale arti- facts	—
Groth16 example with WHIR (ZK stages)	all stages suc- ceed; gnark wrap blocked on stale arti- facts	—
Phase 2a (zerocheck)	shipped	5/5 passing
Phase 2b (LogUp-GKR sumcheck reduction)	shipped	9/9 passing
Phase 2c (single-column late-binding adapter)	shipped	2/2 passing
Phase 2c (multi-column late-binding adapter)	shipped	2/2 passing
Phase 2c (LateBindingCapable trait + KB impl)	shipped	—
Phase 2c (full multi-chip wiring simulation)	shipped	1/1 passing
Phase 2c (JSON serialisation + round-trip test)	shipped	1/1 passing
Phase 2c (interleaved-codeword optimisation, VEIL Lemma 4.7)	designed; im- plementation deferred	—
Prover wiring (prover.rs try_compute_late_binding_proofs)	shipped (TypeId dis- patch, dual commit, dis- patches both KoalaBearPoseidon2 and ...Inner)	—
Verifier wiring (verifier.rs try_verify_late_binding_proofs)	shipped (TypeId dispatch, mirroring fresh chal- lenger)	—
End-to-end smoke test (fibonacci with ZIREN_USE_WHIR=1)	verified: 19 chips, 29.5 MB late-binding bytes, 32.6 s prover, veri- fier accepts	—
Cross-binding (FRI ↔ WHIR consistency)	not started; soundness gap docu- mented	—
Recursion circuit (verify_whir_pcs)	not started	—
gnark BN254 wrap artifact regeneration	blocked on VK map regen	—

4 Plonky3 Upgrade

The recursion circuit needed several fixes to work with the new upstream FRI verifier. All fixes are committed on `feat/upgrade-plonky3`.

4.1 Per-round PoW witnesses

Upstream's new FRI requires the prover to grind a proof-of-work witness once per round (not just at the final query stage). The recursion-circuit FRI verifier (`crates/recursion/circuit/src/fri.rs`) was missing the per-round `check_witness` call. Added.

4.2 Log-arity observations and opened-values restoration

The new FRI verifier observes the log-arity of each round and absorbs the opened-values vector into the transcript before sampling the next fold randomness. Both observations were stripped in an earlier "compatibility" patch and were restored in `p3-whir` commit `715fdb5`. The recursion circuit mirrors them.

4.3 `check_witness` early-return for zero PoW bits

`GrindingChallenger::check_witness` returns `true` when the requested bits is zero. The recursion circuit's variable-arity version was running the hash anyway; aligned to native behaviour.

4.4 `is_div_active` preprocessed column

The recursion compiler emits `DivF/DivE` instructions in both branches of `Select`, including dead branches where the divisor is zero. Native execution handles this gracefully because `mult==0` skips the operation. The AIR didn't. Added a preprocessed column `is_div_active = is_div AND mult_active` (where `mult_active` marks active alu rows) and gated the division constraint on it. Mirror runtime guard added in `crates/recursion/core/src/runtime/mod.rs`.

4.5 Result

Compressed proofs for fibonacci and other benchmarks verify with the upgraded recursion circuit. Aggregation runs through `prove_core` $\rightarrow$ `compress` $\rightarrow$ `shrink` $\rightarrow$ `wrap_bn254` successfully; the gnark BN254 wrap fails with witness-count mismatch because the gnark circuit artifacts predate the FRI changes (250 extra witness elements; see §11).

5 Soundness Fix Design: Making WHIR Mode Production-Sound

5.1 Current State

The WHIR fast path (`ZIREN_USE_WHIR=1`) in `crates/stark/src/prover.rs` skips transition-constraint verification entirely. The core prover:

1. Commits to the main trace only (no permutation trace, no quotient).
2. Opens the main trace at ζ via WHIR (Jagged).
3. Sets `ShardCommitment.permutation_commit = None` and `quotient_commit = None`.

4. Leaves `ChipOpenedValues.permutation` and `quotient` empty.

The recursion verifier (both native and circuit) short-circuits when `permutation_commit.is_none()`: it skips the PCS round for permutation/quotient, skips `verify_constraints`, and only checks:

- PCS opening of preprocessed + main.
- $\sum_i \text{local_cumulative_sum}_i = 0$.

5.2 What This Leaves Unchecked

Given a committed main trace M and cumulative-sum extraction from the last 14 base-field columns of M , the verifier does *not* check:

AIR check	FRI mode	WHIR fast path
$\sum \text{lookup events} = 0$ (permutation argument)	✓ via quotient check	✓ via cumulative sum
Transition: $C(M[i], M[i+1], P[i]) = 0 \forall i$	✓ via quotient check	× NOT CHECKED
Boundary: $C_{\text{first}}(M[0]) = 0, C_{\text{last}}(M[n-1]) = 0$	✓ via Lagrange selectors	× NOT CHECKED
Public values = $M[0][\text{pub.cols}]$	✓ via boundary constraint	× NOT CHECKED

Table 1: What is verified in each mode.

A malicious prover can therefore produce a “verifying” proof for a main trace that does not come from any valid execution of the MIPS program, forge public outputs, or violate transition constraints. Only the global septic digest (extracted from the last 14 columns) is bound, and that is bindable separately to memory-init/finalize events.

The current WHIR mode is a benchmark prototype, not a zk-proof.

5.3 Target Soundness Design

The fix must re-introduce transition-constraint verification while preserving the Jagged + WHIR performance characteristics. There are three standard options.

Option A — Keep the quotient polynomial, batch it into WHIR. Commit to the quotient polynomial alongside the main trace, open both at ζ , check $q(\zeta) \cdot Z_H(\zeta) = \text{folded_constraints}(\zeta)$. *Pro*: minimal change. *Con*: defeats most of the WHIR speedup — quotient width is $2^{\log\text{-quotient_degree}}$ ($= 2$ in Ziren), each column must be computed, committed, and opened. The tendermint $4.5\times$ speedup would collapse to $\sim 1.3\times$. *Status*: not recommended.

Option B — Logup-GKR plus sumcheck-based constraint verification (SP1 hypercube model). Replace both the permutation argument and the quotient polynomial with GKR / sumcheck reductions:

1. **Constraints $\rightarrow$ sumcheck.** The transition-constraint polynomial $C(X) = \alpha \cdot C_0(X) + \alpha^2 \cdot C_1(X) + \dots$ is viewed as a multilinear polynomial over the Boolean hypercube $\{0, 1\}^{\log n}$. The prover proves

$$\sum_{x \in \{0,1\}^{\log n}} \text{eq}(r, x) \cdot C(x, x_{\text{next}}) = 0$$

via sumcheck ($\log n$ rounds, ~ 3 field elements per round).

2. **Lookups $\rightarrow$ Logup-GKR.** The lookup identity becomes a GKR circuit whose final claim is a multilinear evaluation of a fraction polynomial. The prover runs a few sumcheck rounds to reduce it to a single evaluation.

3. **Final opening.** All final claims are batched and opened at one multilinear point via WHIR.

Pro: matches SP1's direction; Logup-GKR has battle-tested designs; preserves most of the WHIR speedup because no extra trace is committed. *Con:* large implementation effort (requires sumcheck-verifier chips inside the recursion circuit). *Status:* recommended (this document's focus).

Option C — Multilinear constraint check via WHIR opening at ζ . Keep the univariate STARK constraint check but express it in multilinear form. The verifier:

1. Samples α, ζ .
2. Opens $\text{main}(\zeta), \text{main}(\zeta\omega), \text{preproc}(\zeta), \text{preproc}(\zeta\omega)$ via WHIR.
3. Locally computes $\text{folded_constraints}(\zeta)$ from the opened values.
4. Asserts $\text{folded_constraints}(\zeta) = 0$.

The last step replaces the identity $q(\zeta) \cdot Z_H(\zeta) = \text{folded}(\zeta)$ with the direct assertion $\text{folded}(\zeta) = 0$. This is only sound if $\text{folded_constraints}(X)$ vanishes on the trace domain, which still requires a sumcheck argument to prove — collapsing Option C into Option B. *Status:* subsumed by Option B.

5.4 Chosen Design: Option B

Protocol sketch (prover)

Let M be the main trace (multilinear, dimension $\log n$ over $\{0, 1\}^{\log n}$), P the preprocessed trace, pub the public values.

Round 1. Commit to M via WHIR (same as today).

Round 2. Sample constraint-mixing challenge α .

Round 3. Constraint sumcheck. Prover and verifier run sumcheck for

$$\sum_{x \in \{0,1\}^{\log n}} \text{eq}(r, x) \cdot \left[\sum_i \alpha^i \cdot C_i(M(x), M(x\omega), P(x), P(x\omega), \text{pub}) \right] = 0,$$

where $r \in \mathbb{F}^{\log n}$ is sampled by the verifier at the start. After $\log n$ rounds, the claim reduces to a single multilinear-value claim on M and P at point $r' \in \mathbb{F}^{\log n}$.

Round 4. Lookup (Logup-GKR). For each lookup table T with events $e_1, \dots, e_k$:

$$\sum_j \frac{1}{\beta - e_j} = \sum_t \frac{\text{multiplicity}(t)}{\beta - t}.$$

The prover runs a GKR circuit whose output wire is 0. The GKR reduction ends with a claim on M, P at another point r'' .

Round 5. Batched WHIR opening. All multilinear claims on M (from sumchecks and GKR) are batched via random linear combination and opened at a single point via WHIR. Same for P .

Verifier cost

- Sumcheck of degree- d polynomial over $\log n$ rounds: $\log n \cdot (d + 1)$ field elements observed, one final multilinear evaluation check.

- Logup-GKR: $O(\log k)$ sumcheck rounds per table.
- WHIR opening: same cost as today’s Jagged + WHIR open.

Total verifier work: $O(\log n \cdot (d + |\text{tables}|))$ — roughly the same asymptotic cost as today, *plus* constraint soundness.

Recursion implications

The recursion circuit currently contains:

- `verify_shard` — iterates over chips, calls `verify_constraints`.
- `verify_constraints` — computes `folded_constraints(ζ)` and $q(ζ) \cdot Z_H(ζ)$ and asserts equality.

Under Option B the recursion circuit needs:

1. A `SumcheckVerify` chip (skeleton already exists in `crates/recursion/core/src/chips/sumcheck_verify.rs`).
2. A `GKRVerifier` circuit (new).
3. A `MultilinearEvalCheck` that reconciles a claimed multilinear evaluation at r with the value opened by WHIR.
4. A replacement for `verify_constraints` that runs `SumcheckVerify` per chip, runs `GKRVerifier` per lookup table, batches all multilinear claims, and checks them against WHIR-opened values.

Migration path

1. **Phase 2a (done)** — sumcheck primitives, hypercube constraint evaluator, `ShardProof::zerocheck_proofs` field, prover + verifier wiring. See Section 7.
2. **Phase 2b** — Logup-GKR for lookup arguments in the native prover. Removes the permutation trace entirely.
3. **Phase 2c** — wire `SumcheckVerify` and `GKRVerifier` into the recursion circuit. Replace `verify_constraints` with the sumcheck-based version.
4. **Phase 3** — formal verification of the sumcheck / GKR verifier chips.

5.5 Projected Performance Impact

Compared to the current (unsound) WHIR fast path:

Operation	WHIR fast path	WHIR + Logup-GKR	Overhead
Main trace commit	same	same	0%
Constraint check	skipped	sumcheck ($\log n$ rounds)	+10–15%
Lookup check	cumulative sum read	Logup-GKR sumcheck	+5–10%
PCS open	one point	batched multi-point	+5%
Total expected	baseline		+20–30%

Table 2: Projected end-to-end overhead of Option B relative to the unsound WHIR fast path.

Projected tendermint performance: $502\text{ s} \cdot 1.25 \approx 630\text{ s}$, still **3.6×** faster than main FRI (2270 s), with full soundness restored.

5.6 Immediate Action Items

1. Rename the current flag to ZIREN_WHIR.UNSAFE=1 with a runtime warning; the existing name misleads about soundness.
2. Complete Phase 2a follow-up: multi-point WHIR opening so the sumcheck final claim $C(r')$ can be checked against an actual opening.
3. Implement Phase 2b (Logup-GKR) to make all chips zerocheck-eligible.
4. Wire into the recursion circuit (Phase 2c).
5. Formal-verification pass (Phase 3).

6 System Design: Proof-System Architecture

This section presents the full proof-system design of the Ziren WHIR pipeline. Where prior sections (§§7–10) present each construction in isolation, here we describe how they compose into a single end-to-end prover/verifier flow, with explicit references to the implementation files.

6.1 Proof-System Overview

A Ziren shard proof attests that one shard of a MIPS program executed correctly. The proof contains:

- A polynomial commitment to the per-chip main traces.
- Sumcheck-based proofs of transition-constraint and lookup soundness.
- Per-row multilinear-extension (MLE) openings of the main trace at GKR-derived points, bound back to the main commit via a late-binding wrapper.
- Cross-binding spot-check evidence linking the FRI commit and the WHIR late-binding commit to the same trace (production target).

The pipeline replaces three FRI-pipeline components with multilinear equivalents:

1. **Quotient identity** ($Q(\zeta) \cdot Z_H(\zeta) = C(\zeta)$) is replaced by zerocheck: $\sum_{\mathbf{b} \in \{0,1\}^m} \mathbf{eq}(\mathbf{r}, \mathbf{b}) \cdot C(\mathbf{b}) = 0$, verified via sumcheck.
2. **Permutation-trace lookup arguments** are replaced by LogUp-GKR with a per-layer sumcheck reduction.
3. **Univariate FRI opening at ζ** is augmented (not replaced — the dual-commit approach keeps both for now) by WHIR multilinear opening at the GKR-derived row coordinates $\mathbf{r}_{\text{row}}$.

6.2 Data Layout: Jagged Polynomial Commitment

Each shard contains N chip traces of differing heights h_c and widths w_c , $c = 0, \dots, N-1$. Naively committing each chip separately requires N merkle trees and N PCS instances. We adopt the Jagged construction [6] to amortise this:

Construction 6.1 (Jagged trace packing). Let $\mathbf{T}_0, \dots, \mathbf{T}_{N-1}$ be per-chip trace matrices. The packed dense vector $\mathbf{q}$ is the column-major concatenation of all per-chip columns:

$$\mathbf{q} = \mathbf{T}_0[:, 0] \parallel \mathbf{T}_0[:, 1] \parallel \dots \parallel \mathbf{T}_{N-1}[:, w_{N-1} - 1].$$

Cumulative offsets t_k satisfy $t_0 = 0$, $t_k = t_{k-1} + h_{c(k)}$ where k ranges over global column indices. Padding brings $|\mathbf{q}|$ to the next power of two 2^n .

The sparse (row, col)-indexed polynomial $p(\mathbf{z}_r, \mathbf{z}_c)$ relates to $\mathbf{q}$'s MLE $\tilde{\mathbf{q}}$ via

$$p(\mathbf{z}_r, \mathbf{z}_c) = \sum_{j=0}^{|\mathbf{q}|-1} \tilde{\mathbf{q}}(j) \cdot \text{eq}(\text{row}(j), \mathbf{z}_r) \cdot \text{eq}(\text{col}(j), \mathbf{z}_c)$$

where $\text{row}(j) = j - t_{\text{col}(j)}$ and $\text{col}(j) = \min_k \{t_k > j\}$. Recovering per-chip evaluations from $\tilde{\mathbf{q}}$ requires a sumcheck argument that we describe in §6.5.

Implementation. Layout is in `crates/stark/src/jagged.rs::pack_traces_jagged`; the resulting `JaggedPacking{dense_values, chip_infos, offsets, total_values, log_dense_size}` carries everything the verifier needs to reconstruct $(\text{row}(j), \text{col}(j))$. The chip-info hash (`hash_chip_infos`) is observed into the challenger before commit to bind chip dimensions to the transcript.

6.3 Polynomial Commitment: WHIR

WHIR [1] is a multilinear PCS based on Reed–Solomon proximity testing. A WHIR commitment to $\mathbf{q} \in \mathbb{F}^{2^n}$ encodes $\mathbf{q}$'s coefficients into an RS codeword on a coset of size 2^{n+r} ($r = \text{log-inv-rate}$, set to 4 in our deployment for 80-bit security) and merkle-commits the row hashes of the codeword matrix.

The WHIR open phase combines:

1. Sumcheck folding rounds (one per WHIR round, each round folds f variables for folding-factor f).
2. STIR queries at K random codeword positions, with per-round merkle paths.
3. Out-of-domain (OOD) sample evaluations to bind the polynomial against the codeword.
4. Per-round proof-of-work (PoW) grinding.
5. A final-round in-the-clear polynomial of degree $2^{\text{final_sumcheck_rounds}}$.

Folding-factor scaling. Our jagged shards have $2^n \in \{2^{22}, \dots, 2^{27}\}$, but KoalaBear's two-adicity caps at 24. We scale the folding factor as $f = \max(4, n + r - 24)$ to keep the folded codeword in the two-adic subgroup. For $n = 27$: $f = 7$, giving $\lceil n/f \rceil = 4$ rounds.

Implementation. WHIR PCS configuration is in `crates/stark/src/whir_config.rs`; the late-binding adapter is in `crates/stark/src/whir_late_binding.rs` (see §6.4).

6.4 Late-Binding WHIR

Motivation. The upstream Plonky3 `MultilinearPcs::commit` adapter takes opening points at commit time, absorbing them into Fiat-Shamir *before* the column-batching challenge α . Our protocol derives the row coordinates $\mathbf{r}_{\text{row}}$ via GKR sumcheck *after* the commit, which the upstream adapter cannot accommodate.

Construction 6.2 (Late-binding WHIR). Phase order:

1. *Domain separator*: observe protocol parameters into the challenger.
2. *Commit phase*: build the RS codeword merkle and observe its root. Sample OOD points; observe their evaluations into the challenger. At this point the `EqStatement` contains only OOD constraints.

3. *(Caller-driven) post-commit point derivation:* GKR/ zerocheck runs against the now-advanced challenger, producing $\mathbf{r}_{\text{row}}$.
4. *Late-binding registration:* for each derived point $\mathbf{p}$ and value v , the prover invokes `add_late_claim( $\mathbf{p}, v$ )`, which evaluates $\tilde{\mathbf{q}}(\mathbf{p}) = v$ (recorded into the EqStatement) and observes v into the challenger. The point is *not* re-observed because it was derived from the post-commit challenger.
5. *Open phase:* run the standard WHIR open over the accumulated EqStatement (OOD claims + late claims).

Soundness. Identical to upstream WHIR. The post-commit ordering does not weaken the proximity guarantee because (a) the merkle commit binds the prover to a specific polynomial *before* late points are sampled, and (b) the sumcheck and STIR-query rounds operate over the same EqStatement that contains both OOD and late claims, with constraint-batching γ sampled *after* all claims are observed.

Multi-column variant. For w columns, the per-column adapter is invoked w times with a shared challenger, with the prover doing all-observes-then-all-opens (and the verifier mirroring). This works but commits w independent merkle trees, which is wasteful at scale. The Jagged + late-binding combined construction in §6.5 reduces this to one merkle per shard.

Implementation. `crates/stark/src/whir_late_binding.rs`, built directly on `p3-whir` internals (`CommitmentWriter`, `Verifier`, `EqStatement`); no upstream patch required. Tests: 24 unit tests covering single-column, multi-column, tampering rejection, and challenger-replay.

6.5 Jagged + Late-Binding Reduction

The Jagged commitment (§6.2) gives one merkle per shard but commits $\tilde{\mathbf{q}}$, not the per-chip column MLEs that LogUp-GKR’s leaf claims need. The reduction:

Construction 6.3 (Jagged sumcheck reduction). Inputs:

- Dense polynomial $\tilde{\mathbf{q}}$ committed via Construction 6.2.
- Per-chip row coordinates $\mathbf{r}_{\text{row},c}$ derived from GKR.
- Per-chip per-column row-MLE values $y_{c,j} = \widetilde{\mathbf{T}_c[:,j]}(\mathbf{r}_{\text{row},c})$ provided by the prover.

Steps:

1. Prover observes all $y_{c,j}$ into the challenger.
2. Verifier samples batching challenge γ .
3. Prover and verifier compute initial claim $T = \sum_{c,j} \gamma^{\text{idx}(c,j)} \cdot y_{c,j}$.
4. Verifier defines the weight polynomial $\tilde{w}(j) = \gamma^{\text{idx}(c,j)} \cdot \text{eq}(\text{row}(j), \mathbf{r}_{\text{row},c})$ for j in chip c column- j ’s range, 0 elsewhere.
5. Run sumcheck on $T = \sum_j \tilde{\mathbf{q}}(j) \cdot \tilde{w}(j)$ over n variables (degree-2 rounds). After sumcheck, the verifier has $\tilde{\mathbf{q}}(\mathbf{z}^*) \cdot \tilde{w}(\mathbf{z}^*) = T_{\text{final}}$ where $\mathbf{z}^*$ is the sumcheck-derived point.
6. Prover supplies $v^* = \tilde{\mathbf{q}}(\mathbf{z}^*)$; verifier independently computes $\tilde{w}(\mathbf{z}^*)$ from the packing metadata, checks $v^* \cdot \tilde{w}(\mathbf{z}^*) = T_{\text{final}}$.
7. v^* is bound to the WHIR commit by registering $(\mathbf{z}^*, v^*)$ as a late claim (Construction 6.2, step 4).

Remark 6.4 (Convention reversal). The sumcheck protocol’s eval point is in last-variable-first order $[r_n, r_{n-1}, \dots, r_1]$ (because pair-consecutive folding binds the last variable first), whereas `p3_multilinear_util::Poly::eval_base` expects MSB-first $[x_0, x_1, \dots, x_{n-1}]$. The $\mathbf{z}^*$ passed to `add_late_claim` is reversed; `verify_jagged_reduction` mirrors this on the verifier side. This was a real integration bug found and fixed during implementation.

Cost analysis. For a fibonacci shard ($N = 19$ chips, ~ 30 avg cols, $n = 27$ packed):

- Commit: 1 merkle on 2^{27+r} codeword vs. 570 merkle on 2^{17+r} .
- Open: 1 WHIR proof on $n = 27$ vs. 570 WHIR proofs on $n = 17$.
- Sumcheck reduction: $n = 27$ rounds, degree-2 polynomials, parallelisable per round.
- Per-shard proof bytes: ~ 488 KB vs. ~ 29.5 MB ($60\times$ smaller).

Per-phase profile (parallelised, fibonacci shard):	WHIR commit on packed dense	1.7 s (1.9%)
	Sumcheck reduction (rayon)	0.16 s (0.2%)
	WHIR open phase	89.4 s (97.9%)

Implementation. `crates/stark/src/jagged_late_binding.rs`, ~ 1200 lines including 11 unit tests.

6.6 Cross-Binding

Threat model. In the dual-commit configuration (FRI at ζ + WHIR via Construction 6.2), a malicious prover could in principle commit to two different traces $\mathbf{T}_A \neq \mathbf{T}_B$ — the FRI side via the existing `pcs.commit` on $\mathbf{T}_A$, the WHIR side via Jagged late-binding on $\mathbf{T}_B$. Both commits would individually verify, but the proof would fail to attest to a single coherent trace.

Construction 6.5 (Brakedown-style cross-binding spot check). After both commits are observed:

1. Sample K random trace positions $i_1, \dots, i_K$ from the post-both-commits challenger ($K = 80$ for ~ 80 -bit soundness against single-position forgery).
2. For each i_j , the prover provides:
 - (a) The trace value $\mathbf{T}[i_j]$.
 - (b) Merkle paths against the FRI MMCS at the codeword position(s) corresponding to i_j .
 - (c) Merkle paths against the WHIR MMCS at the codeword position corresponding to i_j .
3. Verifier checks both paths via `Mmcs::verify_batch` and that both reveal the same value.

Soundness. If $\mathbf{T}_A \neq \mathbf{T}_B$, they differ at ≥ 1 position; K random positions catch the divergence with probability $1 - (1 - 1/N)^K \approx 1 - 2^{-K}$ for $N = 2^{17}$, $K = 80$.

Status. Both sides fully implemented at host level: prover populates FRI- and WHIR-side merkle paths (`populate_fri_paths`, `prove_cross_binding_spot_check`); verifier checks both via `Mmcs::verify_batch` (`verify_cross_binding_spot_check_fri`, `verify_cross_binding_spot_check_whir`). The verifier dispatcher `verify_cross_binding_for_kb` extracts the WHIR merkle root from the jagged bundle’s `WhirProof.initial_commitment` and reconstructs the codeword shape via the public helper `whir_codeword_dims(log_dense_size)`, which mirrors `make_chip_late_binding_pcs`’s folding-factor derivation. Tamper rejection at the merkle layer is validated by `cross_binding_whir_rejects_tampered_path`.

Implementation. Helpers in `crates/stark/src/jagged_late_binding.rs` (`populate_fri_paths`, `verify_cross_binding_spot_check_fri`, `verify_cross_binding_spot_check_whir`, `prove_cross_binding_spot_check`); the codeword-dimension helper `whir_codeword_dims` in `crates/stark/src/whir_late_binding.rs`; prover wiring in `crates/stark/src/prover.rs` (`try_compute_cross_binding_fri`, `prove_jagged_late_binding_with_cross_binding`); verifier wiring in `crates/stark/src/verifier.rs` (`try_verify_cross_binding`, `verify_cross_binding_for_kb`).

6.7 Soundness Layers Summary

The combined pipeline composes the following soundness layers:

Layer	Mechanism	Soundness Bound
Transition constraints	Zerocheck (sumcheck)	$d \cdot m / \mathbb{F}_q $
Lookup arguments	LogUp-GKR + per-layer sumcheck	$3 \cdot \log_2 N / \mathbb{F}_q $
Row-MLE binding to commit	Late-binding WHIR	WHIR proximity
FRI $\leftrightarrow$ WHIR coherence	Cross-binding spot-check (C.1)	$1 - (1 - 1/N)^K$

For our parameters ($|\mathbb{F}_q| \approx 2^{124}$, $m \leq 27$, $d = 2$, $N \leq 2^{27}$, $K = 80$), each layer’s soundness error is $\leq 2^{-80}$.

6.8 Implementation Activation Matrix

Configuration	Env flags
FRI (default, unchanged)	<i>(none)</i>
WHIR fast path + per-chip late-binding	<code>ZIREN_USE_WHIR=1</code>
WHIR fast path + jagged late-binding + cross-binding (FRI + WHIR sides)	<code>ZIREN_USE_WHIR=1</code> <code>ZIREN_LATE_BINDING=jagged</code> <code>+ ZIREN_CROSS_BINDING=1</code>

Production deployment should use the fourth row (jagged late-binding with cross-binding). End-to-end smoke tests on fibonacci, JSON, keccak, and large-sum (multi-shard) workloads validate the first three rows; the fourth row’s merkle-layer correctness is validated by lib-level round-trip and tamper-rejection unit tests against the same MMCS instance the prover uses.

7 Phase 2a: Zerocheck Integration

Phase 2a delivers the sumcheck infrastructure that replaces the quotient polynomial in the WHIR fast path. The integration is complete and end-to-end verified for fibonacci; the remaining soundness gap (transition constraints not yet bound to a PCS opening) is scheduled for the follow-up phase.

7.1 What Ships

Sumcheck core (`crates/stark/src/zerocheck_prover.rs`, **5 unit tests, all passing**).

- `eq_mle_table(r)` – dense evaluation of `eq(r, ·)` over $\{0, 1\}^m$.
- `MultilinearExt::`{`fold_first`, `evaluate`} – standard MLE operations.
- `prove_zerocheck_multilinear(c_evals, r)` – deterministic prover.
- `verify_zerocheck_rounds(proof, r, initial_claim)` – deterministic verifier.

- `prove_zerocheck_with_challenger(c_evals, m, challenger)` – Fiat-Shamir prover.
- `verify_zerocheck_with_challenger(proof, m, initial_claim, challenger)` – Fiat-Shamir verifier.

Hypercube constraint evaluator. `eval_constraints_on_hypercube` iterates each row of the Boolean hypercube, lifts the (base-field) main and preprocessed rows to the extension field, and invokes the chip’s `Air::eval` via `VerifierConstraintFolder`. Produces a length- 2^m table of extension-field values representing

$$C_{\text{batched}}(b) = \sum_j \alpha^j \cdot C_j(\text{row}_b, \text{row}_{b+1}, \text{preproc}_b, \text{preproc}_{b+1}, \text{pub}).$$

Selectors are set analytically per row (`is_first_row`, `is_last_row`, `is_transition`); next-row accesses use cyclic wrap within the hypercube.

ShardProof::zerocheck_proofs field. New optional field serialized with `#[serde(default)]` for backward-compatibility:

```
pub struct ShardProof<SC: StarkGenericConfig> {
    // existing fields
    #[serde(default)]
    pub zerocheck_proofs: Option<Vec<ZerocheckProof<Challenge<SC>>>>,
}
```

Prover wiring (WHIR fast path). After sampling α :

- If `chip.permutation_width() > 0` (lookup-dependent chips): emit an empty placeholder proof. These chips become eligible for zerocheck once Phase 2b removes the permutation argument.
- Otherwise: `run eval_constraints_on_hypercube and prove_zerocheck_with_challenger.`

Timing is logged to `/tmp/ziren.open_breakdown.txt` under `ZEROCHECK total=...ms`.

Verifier wiring. After sampling α , if `zerocheck_proofs` is present, for each chip/proof pair the verifier replays the sumcheck transcript. Empty `rounds` means “skipped” (permutation-dependent chip). Non-empty proofs are passed to `verify_zerocheck_with_challenger`. New error variants: `ZerocheckFailed`, `InvalidProofShape`.

Downstream fixes. `ShardProof` literals updated in `crates/sdk/src/provers/mock.rs` and `crates/recursion/circuit/src/stark.rs`.

7.2 End-to-End Test Results

Mode	Result	Notes
FRI (quotient + permutation)	PASS (8.1 s)	Existing pipeline unchanged
WHIR core (zerocheck emitted, skipped for all 19 chips)	PASS (6.8 s)	Serialization round-trip works
WHIR compressed	PASS (6.9 s)	Recursion verifier bypasses empty p

Table 3: Phase 2a end-to-end verification on fibonacci.

On fibonacci, all 19 CPU-program chips have `permutation_width() > 0`, so `ZEROCHECK total=0ms chips=19` — the sumcheck is never actually executed for this workload. Real-world exercise of the zerocheck math will come once Phase 2b removes the permutation argument.

7.3 Phase 2a Scope Limitation (By Design)

Chips that use the lookup permutation argument are skipped because their `Air::eval` calls `builder.permutation()`, which requires a permutation trace we don't have in the WHIR fast path. Two ways to lift this:

1. **Phase 2b** (LogUp-GKR): replace the permutation argument with GKR. Chips no longer need `builder.permutation()`. All chips become zerocheck-eligible.
2. **Interim fallback**: generate the permutation trace in the WHIR fast path purely for the zerocheck eval (not committed).

Phase 2b is the correct long-term fix and matches SP1's hypercube architecture.

7.4 Files Changed (Phase 2a)

<code>crates/stark/src/lib.rs</code>	+1 module declaration
<code>crates/stark/src/zerocheck.rs</code>	+1 derive Serialize/Deserialize
<code>crates/stark/src/zerocheck_prover.rs</code>	+420 NEW
<code>crates/stark/src/types.rs</code>	+5 ShardProof field
<code>crates/stark/src/prover.rs</code>	+55 WHIR path wiring
<code>crates/stark/src/verifier.rs</code>	+40 zerocheck verify + errors
<code>crates/sdk/src/provers/mock.rs</code>	+1
<code>crates/recursion/circuit/src/stark.rs</code>	+1

8 Phase 2b: LogUp-GKR Protocol Core

Phase 2b Step 1 delivers a complete, unit-tested LogUp-GKR fraction-sum protocol in `crates/stark/src/logup_gkr.rs`. The protocol takes a set of leaf fractions $\{(\text{num}_i, \text{denom}_i)\}$ and lets the prover commit to the claim

$$\sum_i \frac{\text{num}_i}{\text{denom}_i} = 0$$

via a logarithmic-depth reduction. Each GKR layer is a *folding* step (not a sumcheck round) because the fraction-combine identity is bilinear in the two halves.

8.1 Fraction-Sum Combine

For two fractions $(a, b) = a/b$ and $(c, d) = c/d$,

$$\frac{a}{b} + \frac{c}{d} = \frac{a \cdot d + b \cdot c}{b \cdot d}.$$

We encode this as the binary operation

$$(a, b) \oplus (c, d) = (a \cdot d + b \cdot c, b \cdot d).$$

The prover builds a balanced binary tree over leaf fractions using $\oplus$; the root is (N_m, D_m) . For a balanced LogUp argument, $N_m = 0$, which the verifier checks in the clear.

8.2 GKR Reduction

At each layer k (from root m down to leaves 0), the current claim is a multilinear evaluation of (N_k, D_k) at a point $z \in \mathbb{F}_{\text{ext}}^{m-k}$. The identity

$$\begin{aligned} N_{k-1}(z||0) \cdot D_{k-1}(z||1) + D_{k-1}(z||0) \cdot N_{k-1}(z||1) &= N_k(z), \\ D_{k-1}(z||0) \cdot D_{k-1}(z||1) &= D_k(z), \end{aligned}$$

lets the prover produce four values $(N_k(z||0), N_k(z||1), D_k(z||0), D_k(z||1))$ that the verifier checks against the current claim via the equations above. The verifier samples $r^* \in \mathbb{F}_{\text{ext}}$ and folds the claim to the new point $z||r^*$:

$$\begin{aligned} N_{k-1}(z||r^*) &= (1 - r^*) \cdot N_{k-1}(z||0) + r^* \cdot N_{k-1}(z||1), \\ D_{k-1}(z||r^*) &= (1 - r^*) \cdot D_{k-1}(z||0) + r^* \cdot D_{k-1}(z||1). \end{aligned}$$

After m layers, the remaining claim is $(N_0(\text{eval_point}), D_0(\text{eval_point}))$ – the multilinear evaluation of the *leaf* fractions at the accumulated point. The caller closes the loop by checking this against the fingerprints (multiplicity _{i} , α – fingerprint _{i}) reconstructed from the main-trace PCS opening at `eval_point`.

8.3 Soundness Tests

Six unit tests validate the protocol:

Test	What it establishes
<code>fraction_combine_is_associative_on_small_input</code>	Algebraic sanity of $\oplus$
<code>sum_of_balanced_fractions_has_zero_numerator</code>	Target identity: balanced sends/receives $\Rightarrow N_m$
<code>logup_gkr_prove_verify_roundtrip_on_balanced_input</code>	Honest prover round-trip
<code>logup_gkr_leaf_claim_matches_mle_evaluation</code>	Reduction identity: final claim = $N_0(\text{eval_point})$
<code>logup_gkr_rejects_tampered_combine_identity</code>	Soundness: corrupted combine round is rejected
<code>logup_gkr_rejects_tampered_root</code>	Soundness: corrupted root is rejected

Table 4: LogUp-GKR unit tests, all passing.

The `leaf_claim_matches_mle_evaluation` test is the key soundness property: without it, the protocol might appear to verify but would not actually bind the prover to the committed fraction leaves.

8.4 Public API

```
pub struct Fraction<EF>      { pub num: EF, pub denom: EF }
pub struct LogUpGkrProof<EF> {
    pub root: (EF, EF),
    pub rounds: Vec<(EF, EF, EF, EF)>, // (n0, n1, d0, d1) per layer
    pub eval_point: Vec<EF>,
    pub leaf_claim: (EF, EF),
}

pub fn build_fraction_tree<EF>(leaves: &[Fraction<EF>]) -> Vec<Vec<Fraction<EF>>>;
pub fn sum_of_fractions<EF>(leaves: &[Fraction<EF>]) -> Fraction<EF>;
pub fn eval_mle<EF>(table: &[EF], r: &[EF]) -> EF;
pub fn eval_mle_first_var<EF>(table: &[EF], r: &[EF]) -> EF;
```

```

pub fn prove_logup_gkr<F, EF, Challenger>(
    leaves: &[Fraction<EF>], challenger: &mut Challenger
) -> LogUpGkrProof<EF>;

pub fn verify_logup_gkr<F, EF, Challenger>(
    proof: &LogUpGkrProof<EF>, challenger: &mut Challenger
) -> Option<(Vec<EF>, Fraction<EF>)>;

```

The `eval_point` returned is in MSB-first order (matching `eval_mle_first_var`), so the caller can evaluate the leaf MLE directly at that point with no re-ordering.

8.5 Integration Path (Phase 2b Steps 2 – 4)

Outstanding work to connect this protocol to the native prover:

1. **Build the leaves from lookup events.** For each chip:
 - Enumerate send/receive events;
 - For each event, compute the fingerprint $f = \beta \cdot \text{payload}_0 + \beta^2 \cdot \text{payload}_1 + \dots$;
 - Leaf numerator = $\pm$ multiplicity (+ for sends, – for receives);
 - Leaf denominator = $\alpha - f$.
2. **Attach the GKR proof to ShardProof.** Add a new optional field `logup_gkr_proofs: Option<Vec<LogUpGkrProof<Challenge<SC>>>>` with `#[serde(default)]`.
3. **Remove the permutation trace in WHIR mode.** Skip `generate_permutation_trace` when LogUp-GKR is enabled. Drop the permutation-commit round from the Fiat-Shamir transcript.
4. **Verifier: close the loop.** After verifying the GKR layers, check the leaf claim against fingerprints recomputed from the opened trace values at `eval_point`.
5. **Unblock zerocheck.** Once the permutation argument is gone, every chip becomes zerocheck-eligible; the Phase 2a limitation at `chip.permutation_width() > 0` can be dropped.

8.6 Combined Soundness Picture

Component	FRI	WHIR (pre-2a)	WHIR (2a only)	WHIR (2a + 2b + multi-point open)
Main trace binding	✓	✓	✓	✓
Lookup integrity	✓	Δ^1	Δ^1	✓(GKR)
Transition constraints	✓	×	Δ^2	✓(sumcheck)
Production-sound?	✓	×	×	✓

Table 5: Soundness status across phases.

¹Only the cumulative-sum balance is checked.

²Sumcheck transcript is bound, but the final claim is not yet checked against a PCS opening.

9 LogUp-GKR: Bug, Fix, and End-to-End Integration

9.1 The Soundness Bug (And Why It Was Hard to Spot)

The initial Phase 2b implementation in `crates/stark/src/logup_gkr.rs` used a folding-only reduction. At each GKR layer, the prover sent four values

$$(N_k(z\|0), N_k(z\|1), D_k(z\|0), D_k(z\|1)),$$

and the verifier checked the combine identity

$$\begin{aligned} N_{k+1}(z) &\stackrel{?}{=} N_k(z\|0) \cdot D_k(z\|1) + D_k(z\|0) \cdot N_k(z\|1), \\ D_{k+1}(z) &\stackrel{?}{=} D_k(z\|0) \cdot D_k(z\|1), \end{aligned}$$

before folding a challenge r^* to obtain the next claim at $z\|r^*$.

This identity is only valid for Boolean z . At a random $z \in \mathbb{F}^{m-k-1}$ the right-hand side is a degree-2 polynomial in each coordinate, while N_{k+1} is multilinear (degree 1). For arbitrary z the identity fails.

Why the original test suite didn't catch it. The honest-roundtrip test used balanced fractions with all denominators drawn from $\{7, 11\}$. Those repeated denominators made the degree-2 cross-terms vanish algebraically, so the check accidentally held. A regression test with varied denominators (`logup_gkr_roundtrip_with_varied_denominators`) fails with the folding-only reduction but now passes with the sumcheck-based fix.

9.2 The Correct Reduction

At each GKR layer we need to reduce a claim $(N_{k+1}(z), D_{k+1}(z))$ into claims on N_k, D_k at a fresh point. The MLE-extended combine identity

$$\tilde{N}_{k+1}(z) = \sum_{b \in \{0,1\}^{m-k-1}} \text{eq}(z, b) \cdot [\tilde{N}_k(b, 0) \cdot \tilde{D}_k(b, 1) + \tilde{D}_k(b, 0) \cdot \tilde{N}_k(b, 1)]$$

is genuinely identical at every z because the product of multilinear functions is only evaluated at Boolean b inside the sum. We turn this into a sumcheck claim.

Protocol per layer

1. **Batch.** Sample $\lambda \in \mathbb{F}_{\text{ext}}$. The merged claim is $\lambda \cdot \tilde{N}_{k+1}(z) + \tilde{D}_{k+1}(z)$.
2. **Sumcheck.** Run $m - k - 1$ rounds of sumcheck over

$$g(b) = \text{eq}(z, b) \cdot (\lambda \cdot (\tilde{N}_k \tilde{D}_k \text{ cross}) + \tilde{D}_k(b, 0) \cdot \tilde{D}_k(b, 1)).$$

g has total degree 3 in each variable being summed out, so each round sends 4 field elements (evaluations at $X \in \{0, 1, 2, 3\}$).

3. **Final evaluation.** The prover sends $(\tilde{N}_k(r^*, 0), \tilde{N}_k(r^*, 1), \tilde{D}_k(r^*, 0), \tilde{D}_k(r^*, 1))$. The verifier checks the sumcheck's final identity directly.
4. **Line protocol.** Sample $t \in \mathbb{F}_{\text{ext}}$ and fold the two pairs of values into a single claim at (r^*, t) .

After m layers, the accumulated point $\mathbf{r} \in \mathbb{F}^m$ and the final claim $(\tilde{N}_0(\mathbf{r}), \tilde{D}_0(\mathbf{r}))$ can be checked against the fingerprints reconstructed from the main-trace PCS opening at $\mathbf{r}$.

9.3 Implementation

Proof shape. The new LogUpGkrProof carries per-layer reductions:

```
pub struct LogUpGkrLayerProof<EF> {
  pub sumcheck_rounds: Vec<[EF; 4]>, // degree-3 round polys
  pub final_evals: [EF; 4],          // (N_k(r*,0), N_k(r*,1), D_k(r*,0), D_k(r*,1))
}

pub struct LogUpGkrProof<EF> {
  pub root: (EF, EF),
  pub layers: Vec<LogUpGkrLayerProof<EF>>,
  pub eval_point: Vec<EF>,
  pub leaf_claim: (EF, EF),
}
```

Variable ordering. The sumcheck folds the last variable first at every round (cache-friendly pair-consecutive layout), so the raw `r_star` vector built during the sumcheck encodes challenges in reverse variable order. The prover and verifier both reverse `r_star` before appending the line challenge, so the returned `eval_point` matches the `eval_mle_first_var` convention used by the caller.

Unit tests. All 8 tests pass:

- `fraction_combine_is_associative_on_small_input`
- `sum_of_balanced_fractions_has_zero_numerator`
- `logup_gkr_prove_verify_roundtrip_on_balanced_input`
- `logup_gkr_leaf_claim_matches_mle_evaluation` (*final claim equals MLE at eval_point*)
- `logup_gkr_roundtrip_with_varied_denominators` (*regression for the folding-only bug*)
- `logup_gkr_rejects_tampered_final_evals`
- `logup_gkr_rejects_tampered_root`
- `logup_gkr_leaf_claim_reconstructs_from_row_mle` (*end-to-end: real chip → leaves → GKR → reconstruction from row-MLE opening matches*)

End-to-end. The WHIR fast path now emits LogUp-GKR proofs per chip and the verifier replays them. Fibonacci core proof: 7.2 s (1.15 kHz); compressed: 7.1 s.

Mode	e2e (s)	kHz	Status
main FRI	12.54	0.66	Baseline
upgrade FRI	8.21	1.01	+53% vs main
upgrade WHIR (zerocheck skipped, no GKR)	6.8	1.22	+85% vs main, unsound
upgrade WHIR + sumcheck LogUp-GKR	7.2	1.15	+74% vs main, transcript-bound GKR

Table 6: Fibonacci core-proof times across configurations. LogUp-GKR adds 5% over the bare WHIR path thanks to the 1.5 s per-shard sumcheck cost. Compared to main FRI, we keep 74% of the measured speedup while restoring lookup-argument binding.

9.4 Closing the Soundness Loop

The `reconstruct_leaf_claim_from_openings` helper (`crates/stark/src/logup_gkr.rs`) implements the final soundness check. Given the opened main-trace and preprocessed-trace values at the row coordinates of $\mathbf{r}$, it evaluates each lookup interaction’s multiplicity and fingerprint expressions at that row point (using `VirtualPairCol::apply::<EF, EF>`), then combines the per-interaction fractions into the leaf-layer MLE values at $\mathbf{r}$:

$$\begin{aligned}\tilde{N}_0(\mathbf{r}) &= \sum_j \text{eq}(\mathbf{r}_{\text{int}}, j) \cdot \text{mult}_j(\mathbf{r}_{\text{row}}), \\ \tilde{D}_0(\mathbf{r}) &= \sum_j \text{eq}(\mathbf{r}_{\text{int}}, j) \cdot (\alpha + \beta \cdot \text{kind}_j + \sum_i \beta^{i+1} \cdot \text{value}_{j,i}(\mathbf{r}_{\text{row}})).\end{aligned}$$

The unit test `logup_gkr_leaf_claim_reconstructs_from_row_mle` builds a real single-column chip with a single lookup, runs the GKR prover, evaluates the row-MLE of the trace at `proof.eval_point` using `eval_mle_first_var`, calls the reconstruction helper, and confirms the result matches `proof.leaf_claim` exactly — closing the protocol’s soundness chain end-to-end on synthetic data.

Outstanding infrastructure. What the verifier still lacks is the ability to *obtain* the main-trace row-MLE values at `proof.eval_point`: today the WHIR fast path opens the trace only at a univariate point ζ . Once the PCS layer exposes multilinear opening at arbitrary points (the same infrastructure needed to close the Phase 2a zerocheck gap), the verifier will:

1. After `verify_logup_gkr`, obtain the opened trace values at `proof.eval_point` via WHIR.
2. Split `eval_point` into `(r_row, r_int)`.
3. Call `reconstruct_leaf_claim_from_openings`.
4. Compare the reconstruction to `proof.leaf_claim`.

All of the arithmetic is already implemented and unit-tested; only the PCS-level plumbing remains.

9.5 Files Touched

<code>crates/stark/src/logup_gkr.rs</code>	rewritten with sumcheck reduction
<code>crates/stark/src/zerocheck_prover.rs</code>	made <code>fold_table_first</code> pub
<code>crates/stark/src/prover.rs</code>	WHIR path emits <code>LogUpGkrProof</code> per chip
<code>crates/stark/src/verifier.rs</code>	WHIR path replays <code>LogUpGkrProof</code>
<code>crates/stark/src/types.rs</code>	<code>ShardProof::logup_gkr_proofs</code> field
<code>crates/sdk/src/provers/mock.rs</code>	literal update
<code>crates/recursion/circuit/src/stark.rs</code>	literal update

10 Phase 2c: WHIR Late-Binding and Per-Chip Row-MLE

Phase 2c closes the soundness gap left by Phases 2a (zerocheck) and 2b (LogUp-GKR): the per-chip `LogUpRowOpening` values are bound cryptographically to the main-trace WHIR commitment. This requires two pieces of new infrastructure: a WHIR PCS adapter that supports *late-binding* evaluation points, and a restructured GKR leaf claim that needs only a single α -combined row-MLE value per chip rather than one per column.

10.1 The Problem with the Standard WHIR Adapter

The upstream `MultilinearPcs::commit(poly, opening_points, ch)` adapter requires the evaluation points at commit time: opening points are absorbed into the Fiat-Shamir transcript *before* the column-batching challenge α is sampled. This binding order is essential for the upstream soundness proof.

Our protocol cannot satisfy that ordering. After committing the main trace we run zerocheck (which derives ζ) and LogUp-GKR (which derives a per-chip evaluation point r_{row}); only then do we know the full set of points at which we want to open the trace. Sampling r_{row} before commit is impossible because GKR’s sumcheck transcript depends on the merkle root.

10.2 Late-Binding Adapter

Insight. WHIR’s commitment phase already adds OOD constraints to the equality statement *after* the merkle root is observed (`committer.commit` in `p3-whir`). The cryptography permits adding constraints between merkle commit and the WHIR rounds; only the upstream API hides this capability behind a one-shot `commit` call.

Implementation. `crates/stark/src/whir_late_binding.rs` (single column):

- `commit_column(poly, ch) -> (commit, prover_state)`
domain separator $\rightarrow$ width $\rightarrow$ merkle commit, observe root $\rightarrow$ OOD sample/observe.
- `add_late_claim(state, point, ch) -> value`
evaluate $p(\text{point})$, add to statement, observe *value* into challenger. Point itself is not re-observed: it is already bound by the caller’s downstream protocol that derived it.
- `open(state, ch) -> (proof, late_claims)`
runs WHIR’s prove phase against the accumulated statement.
- `replay_commit_into_challenger(proof, ch) -> parsed`
verifier-side: re-runs the commit-phase challenger interactions to reach the same state the prover had.
- `finish_verify(proof, parsed, late_claims, ch) -> Result`
verifier-side: builds the equality statement in the same order the prover did (OOD first, then late claims), passes an empty-OOD `ParsedCommitment` to suppress the internal `concatenate` (which would re-add OOD constraints in the wrong position and corrupt γ -batching).

Soundness validation. Two unit tests in `whir_late_binding.rs`:

- `late_binding_round_trip_single_column`: end-to-end commit $\rightarrow$ derive point $\rightarrow$ add claim $\rightarrow$ prove $\rightarrow$ verify with prover and verifier sampling the late point from their own challengers (sanity-checked via `assert_eq!`).
- `late_binding_rejects_tampered_value`: confirms the binding is cryptographic — a tampered late-claim value is rejected by `finish_verify` (returns `Sumcheck(InvalidPowWitness)`).

Constraint ordering subtlety. `combine_hypercube` weights the i -th constraint by γ^i . The prover’s statement is `[ood0, ..., oodN-1, late0, ..., lateL-1]`, set by `CommitmentWriter.commit` (OOD first) then `add_late_claim` (late after). The naive verifier would call `Verifier::verify(parsed, public_statement = late_claims)`, which internally does `statement.concatenate(&parsed.ood)` and ends up with `[late, ood]` — different γ^i -weights,

transcript mismatch, sumcheck rejects. Our `finish_verify` builds the full `[ood,late]` statement explicitly and passes an empty-`OOD ParsedCommitment` to neutralise the internal concatenation.

10.3 Architectural Choice for Multi-Column Binding

Each chip’s main trace has w columns. We need to bind the per-column row-MLE evaluations $v_j = \text{MLE}(\text{col}_j)(r_{\text{row}})$ in the chip’s `LogUpRowOpening` to the WHIR commitment of the chip’s trace.

Two viable approaches:

(A) Per-column WHIR commits. Commit each of the w columns separately, run the late-binding adapter w times. Soundness clean: each v_j is bound by its own WHIR opening at r_{row} . Cost: $w \times \#\text{chips}$ merkle trees and WHIR proofs. For a typical RISC-V STARK ($w \approx 30$, $\#\text{chips} \approx 100$) this is ~ 3000 WHIR commits per shard — a $30\times$ overhead vs the existing single-batch commit.

(B) Single α -combined commit + restructured GKR. Commit each chip’s trace as a single multi-column matrix. WHIR internally batches columns via $g_c = \sum_{j=0}^{w-1} \alpha_c^j \cdot \text{col}_{c,j}$ where α_c is sampled at commit time for chip c . Restructure LogUp-GKR so its leaf-claim reconstruction needs only the *combined* row-MLE $V_c := g_c(r_{\text{row}}) = \sum_j \alpha_c^j \cdot v_{c,j}$ rather than the individual $v_{c,j}$. Cost: one WHIR commit per chip, matching the existing structure.

Why (B) is sound. The malicious-prover attack against the naive multi-column late binding is: prover sees α_c before r_{row} , then chooses per-column values $v_{c,j}$ to satisfy $\sum_j \alpha_c^j \cdot v_{c,j} = g_c(r_{\text{row}})$, gaining $w - 1$ degrees of freedom. Approach (B) eliminates this attack by *never trusting the prover with per-column values*: only the combined value V_c is needed for verification, and V_c is bound directly by WHIR’s opening proof. No $w - 1$ slack exists because there is no w -dimensional unknown.

Why we choose (B). Soundness is identical to (A); cost is $30\times$ smaller; the existing chip commit shape carries through with no per-column explosion. The price is a non-trivial change to the LogUp-GKR leaf-claim reconstruction.

10.4 Restructured Leaf-Claim Reconstruction

Current (Phase 2b) leaf-claim shape (`logup_gkr.rs`):

$$(N_{\text{leaf}}, D_{\text{leaf}}) = \bigoplus_i (m_i, \alpha - f_i(\text{row}))$$

where m_i is the multiplicity and f_i is the fingerprint of the i -th interaction at row `row`. Each f_i is a polynomial in the per-column values $v_{c,j} = \text{MLE}(\text{col}_{c,j})(r_{\text{row}})$ of the chip’s main and preprocessed columns (typically $f_i(\text{row}) = \beta + \text{kind}_i + \sum_k \beta^k \cdot v_{c,j_{i,k}}$).

The reconstruction `reconstruct_leaf_claim_from_openings` currently:

1. Reads $v_{c,j}$ from `LogUpRowOpening.main_at_r_row` and `preproc_at_r_row`.
2. For each interaction i , computes f_i from the relevant $v_{c,j}$.
3. Folds the L interactions through the fraction-sum tree to get $(N_{\text{leaf}}, D_{\text{leaf}})$.

Restructured (Phase 2c) leaf-claim shape:

$$V_c := \sum_{j=0}^{w_c-1} \alpha_c^j \cdot v_{c,j} \quad (\text{single per-chip combined value, bound by WHIR})$$

$$V_c^{\text{prep}} := \sum_{j=0}^{w'_c-1} \alpha_c^{w_c+j} \cdot v_{c,j}^{\text{prep}} \quad (\text{combined preprocessed, same chip's } \alpha_c)$$

The reconstruction now needs to compute f_i given only V_c and V_c^{prep} (and the public per-interaction shape). This is impossible in general: f_i depends on *specific* columns $v_{c,j_{i,k}}$, not their α_c -mix.

10.4.1 The Fix: Fingerprint Re-Batching at GKR Time

Restructure the GKR leaves so each leaf ℓ is itself a single β -combined value carrying enough information that the verifier can reconstruct it from V_c alone.

Concretely, fold the per-interaction fingerprint into the column batching: instead of computing f_i from individual $v_{c,j}$, we choose the column-batching coefficients such that

$$f_i = \beta_i \cdot V_c + (\text{publicterms})$$

where β_i is fixed by the protocol's interaction shape and is the same on both sides. The challenge is that different interactions i touch different subsets of columns; mapping all of them into a single V_c requires redefining the column-batching coefficients per-interaction.

Concrete construction. For chip c with L_c interactions and total column count $w_c + w'_c$ (main + preprocessed), define:

- Per-interaction column-projection vectors $\pi_i \in \mathbb{F}^{w_c+w'_c}$ where $\pi_i[j] = \beta^k$ if interaction i uses column j in slot k , else 0.
- A single chip-wide combined polynomial $g_c(\mathbf{x}) = \sum_i \gamma^i \cdot \langle \pi_i, \text{cols}(\mathbf{x}) \rangle$ where γ is sampled from the post-merkle challenger and $\text{cols}(\mathbf{x})$ is the column vector of the chip evaluated at $\mathbf{x}$.
- Bind $g_c(r_{\text{row}})$ via WHIR late-binding.

The verifier reconstructs the leaf claim from a *single* $g_c(r_{\text{row}})$ value. The per-interaction fingerprint contributions are stitched together via γ^i binding, which is sampled *after* r_{row} is known — so all degrees of freedom in per-column values are eliminated by the γ -randomness.

Soundness sketch. A malicious prover wishing to lie about a leaf-claim value would need to produce a g_c such that $g_c(r_{\text{row}})$ matches a fabricated leaf-claim. Because γ is sampled after r_{row} and after the per-interaction projections are fixed, the prover has no degrees of freedom: the malicious $g_c(r_{\text{row}})$ would have to equal a specific value over γ -randomness, which holds with probability $1/|\mathbb{F}|$ per column. Across w_c columns, soundness error is $w_c/|\mathbb{F}|$ — negligible for $|\mathbb{F}| > 2^{124}$.

10.5 The VEIL Multilinear PCS Construction (the actual answer)

After the dead end above, a survey of the SP1 VEIL paper [2] (Dalal–Hemo–Rabinovich–Rothblum, slop/crates/veil/paper/veil.pdf) revealed Lemma 4.7 (ZK Multilinear PCS), which is exactly the late-binding multi-column construction we need. The ZK aspect is incidental — the underlying late-binding wrapper composes with any Reed–Solomon-based base PCS, including WHIR, and remains sound for the non-ZK case (we drop the masking padding $k = 0$).

Construction overview. Let $f \in \mathbb{F}^{2^n}$ be the evaluation vector of the n -variable multilinear polynomial we want to commit (e.g. the chip trace serialised in row-major order). Pick a stacking parameter $0 < p < n$.

Commit phase.

1. Decompose f into 2^p stacks f_ℓ of length 2^{n-p} (f_ℓ is the slice where the leading p coordinates equal ℓ). In our STARK use case $2^p = \text{number of chip columns}$ and $2^{n-p} = \text{trace height}$.
2. (Optional ZK pad: append k random elements to each f_ℓ and a fresh row f_{2^p+1} . We use $k = 0$.)
3. Encode each stack: Cf_ℓ via the base RS code.
4. Send the interleaved codeword $\mathfrak{C}_{\ell,i} = (Cf_\ell)_i$ as the merkle commit.

Crucially, no α -batching challenge is sampled at commit time, and no opening points are absorbed yet. The merkle commit is purely structural over the per-column codewords.

Open at a post-commit point $x = (x^{(1)}, x^{(2)})$. $x^{(1)} \in \mathbb{F}^p$ is the column-index dimension, $x^{(2)} \in \mathbb{F}^{n-p}$ is the row dimension (in our case $x^{(2)} = r_{\text{row}}$ from GKR).

1. Prover exposes per-stack values $y_\ell := f_\ell(x^{(2)})$ for $\ell \in \{0, 1\}^p$. These are the per-column row-MLE values — exactly what **LogUpRowOpening** carries.
2. Verifier samples $\vec{\rho} \leftarrow \text{PG}$ *after* seeing the exposed y_ℓ . This is the binding step: $\vec{\rho}$ is the column-batching randomness, sampled post-late-point and post-exposure.
3. Prover sends explicit g' , the linear combination $g'(\cdot) := \sum_\ell \rho_\ell f_\ell(\cdot)$ over the coordinate dimension.
4. Verifier defines a virtual oracle $\mathfrak{C}_\rho$ via $\mathfrak{C}_{\rho,i} = \sum_\ell \rho_\ell \mathfrak{C}_{\ell,i}$, which can be resolved by querying the merkle-committed $\mathfrak{C}$.
5. Run the base evaluation PCS (**this is where WHIR slots in**) to prove $g(x^{(2)}) = a$ where $a := \sum_\ell \rho_\ell y_\ell$ is the verifier-recomputable target.
6. Verifier checks:
 - (a) $\hat{Y}(x^{(1)}) = y$ where $\hat{Y}$ is the multilinear extension of $Y : \{0, 1\}^p \rightarrow \mathbb{F} : \ell \mapsto y_\ell$ and y is the user-claimed evaluation. This binds the per-column values to the requested point in the column dimension.
 - (b) $\sum_\ell \rho_\ell y_\ell = a$ (the explicit linear check on exposed values).

Why this works (and the previous design didn't). The fatal gap in the naive multi-column late-binding was that α (column-batching) had to be sampled at commit time, before the late opening point was known. A malicious prover, knowing α and the eventual late point r_{row} , could pick per-column values v_j to satisfy $\sum_j \alpha^j v_j = g(r_{\text{row}})$, gaining $w - 1$ degrees of freedom.

VEIL Lemma 4.7 inverts the order:

1. Commit *without* any batching challenge.
2. Late-derive r_{row} .
3. Prover *exposes* per-column values y_ℓ first.
4. Verifier samples $\vec{\rho}$ *after* seeing both r_{row} *and* the y_ℓ .

5. The check $\sum_{\ell} \rho_{\ell} y_{\ell} = a$, where a is bound by the base PCS opening of $g(r_{\text{row}})$, makes any single-column lie detectable with probability $1 - 1/|\mathbb{F}|$ over $\vec{\rho}$.

The verifier can compute a from $\vec{\rho}$ and the exposed y_{ℓ} ; the base PCS proof binds $g(r_{\text{row}}) = a$ to the merkle commitment via the virtual oracle $\mathfrak{C}_{\rho}$. All per-column values are bound.

Cost analysis (Lemma 4.7, $k = 0$).

- Commit time: $(2^p + 1) \cdot T_C(2^{n-p})$ — one RS encode per column.
- Prove time: $T_{P_0}(2^{n-p}) + O(2^n)$ — base WHIR proof on a single 2^{n-p} -size polynomial plus the linear combination.
- Verify time: $T_{V_0}(2^{n-p}) + O(k_{0,n-p} \cdot 2^p)$.
- Message complexity: $2^p + 2$ field elements (the exposed y_{ℓ} values plus base PCS messages).

For a typical chip with $w \approx 32$ columns and $h \approx 2^{17}$ rows ($n = 22$, $p = 5$, $n - p = 17$):

- Commit: 33 RS encodes of 2^{17} values — comparable to the existing batched FRI commit (which encodes the full 32×2^{17} matrix as a single codeword anyway).
- Prove: one WHIR proof of size $\sim$ few KB plus ~ 32 extra field elements.
- Verify: standard WHIR verify cost plus ~ 32 coefficient operations.

The overhead vs. plain WHIR is the $2^p \approx 32$ exposed values plus a small linear-algebra check — negligible compared to the $\sim 30\times$ cost of per-column commits (option A) or the research-grade WHIR variant required for option B.

Mapping to our STARK pipeline.

- Per chip, the main trace matrix is w columns by h rows, i.e. an evaluation vector of size $w \cdot h$ over $n = \log_2(w) + \log_2(h)$ variables. Set $p = \log_2 w$.
- Commit via the interleaved codeword (replaces the existing WHIR fast-path `pcs.commit`).
- After zerocheck and LogUp-GKR derive r_{row} , run the open-at-point protocol with $x^{(2)} = r_{\text{row}}$ and $x^{(1)} =$ the p coordinates that select the column index. The exposed y_{ℓ} values are exactly `LogUpRowOpening.main_at_r_row`.
- The verifier reconstructs the leaf claim from the bound y_{ℓ} exactly as the existing `reconstruct_leaf_claim_from_openings` does — no GKR restructuring needed.

10.6 Cross-Binding Plan (Closing the First-Iteration Gap)

The current shipped prover/verifier wiring uses a fresh challenger for the late-binding step, leaving the WHIR commit independent of the FRI commit at ζ . Concrete attack: a malicious prover commits trace A via FRI (used by zerocheck and the existing `pcs.verify`) and trace $B \neq A$ via WHIR late-binding (used to bind `LogUpRowOpening.main_at_r_row`). The verifier accepts both because:

- zerocheck checks A 's transition constraints on its hypercube;
- GKR + late-binding check B 's lookups balance.

A program execution requires *both* for the same trace; the gap lets the prover decouple them.

Two viable closures (in increasing scope). (C.1) Brakedown-style spot checks (smaller change). After both commits are made, sample K random trace positions $i_1, \dots, i_K \in \{0, \dots, 2^h - 1\}$. For each position i_j :

- Prover provides $\text{trace}[i_j]$ (a length- w vector of per-column values).
- Prover provides merkle paths against the FRI commit’s codeword leaves at the codeword position(s) corresponding to i_j (the FRI commit is on the LDE codeword; recovering $\text{trace}[i_j]$ from the codeword requires the evaluation domain to include i_j as a coset point, which it does for the natural-domain coset).
- Prover provides merkle paths against each per-column WHIR commit at the codeword position corresponding to i_j .

Verifier checks: both sides give the same $\text{trace}[i_j]$ for each i_j . Soundness: if $A \neq B$ they differ at ≥ 1 trace position; K random positions catch the divergence with probability $1 - (1 - 1/N)^K$. For $K = 80$, $N = 2^{17}$, the missed-detection probability is $\sim 2^{-80}$.

Cost: $K \cdot \log_2(N \cdot \text{rate})$ extra hashes per side per chip. For typical chips ($K = 80$, $N = 2^{17}$, $\text{rate} = 16$): ~ 1700 hashes per chip per side, so ~ 100 kB extra per chip per shard. Manageable.

C.1 implementation reality check. The Brakedown-style spot-check sounds simple but hits a real impedance mismatch: the FRI commit’s MMCS opens at LDE-codeword positions on a multiplicative-subgroup coset (rate 2 typically); the WHIR commit’s MMCS opens at RS-codeword positions on a different coset (rate 16 in our config). “Trace position i ” is one merkle path in each, but the literal merkle leaves are codeword values that interpolate to $\text{trace}[i]$, not $\text{trace}[i]$ itself. Direct spot-check at canonical trace positions requires:

- Mapping $\text{trace}[i]$ to (FRI codeword position, WHIR codeword position) via the respective coset rotation / RS encoding.
- Querying both MMCS at those positions and de-rotating.
- Re-checking interpolation consistency.

This is implementable but requires touching internal PCS plumbing in both `p3-fri` and `p3-whir`.

C.1 WHIR-side: shipped.

- `CrossBindingSpotCheckProof` wire format
- `prove_cross_binding_spot_check(dense, whirl_state, ch, K)` opens the WHIR commit’s MMCS at K spot-check positions via `Mmcs::open_batch` and serialises the `BatchOpening` per position.
- `verify_cross_binding_spot_check_whir(proof, commit, height, width)` runs `Mmcs::verify_batch` per position and returns true iff every WHIR-side merkle path validates against the parsed WHIR commitment.
- Cryptographic tamper rejection: mutating one merkle path byte causes `verify_cross_binding_spot_check_whir` to return false (test `cross_binding_whir_rejects_tampered_path`).

C.1 FRI-side: not yet started. The FRI commit’s MMCS lives inside `StarkConfig::Pcs`’s prover data (`ShardMainData::main_data`); accessing it for spot-check generation requires either:

- Exposing the underlying MMCS from `p3-fri`’s `TwoAdicFriPcs`, OR
- Threading the MMCS through the `StarkConfig` generic chain.

Both are several-hour PRs on the FRI side. Status: deferred pending the architectural choice between completing C.1 (FRI plumbing) and landing C.2 (drop FRI in WHIR fast path; WHIR commit becomes sole binding).

(C.2) Single-commit migration (cleaner, larger change). Drop the FRI commit entirely from the WHIR fast path. The WHIR late-binding commit becomes the sole binding for the main trace. Required changes:

- In `prover.rs::open`, skip `pcs.commit + pcs.open` for the main trace; set `opening_proof` to a placeholder, set `ShardOpenedValues.chips[*].main` to empty.
- In `verifier.rs::verify_shard`, skip `config.pcs().verify(...)` when WHIR fast path is in use; rely entirely on the late-binding verification for main-trace soundness.
- Add a second late-binding opening at *zerocheck*'s $r_{\text{zerocheck}}$ to bind the zerocheck final claim to the trace (currently the zerocheck final claim is transcript-bound but not PCS-bound; opens an analogous gap to the GKR one).

Soundness: clean (one commit, all openings against it). Cost: no extra commit (saves the FRI commit + open time). Risk: changes the on-wire `ShardProof` shape and the recursion-circuit verifier.

Recommendation. Ship (C.1) first as a bounded incremental fix that closes the soundness gap with the current architecture; (C.2) is the long-term target once the rest of the WHIR fast path stabilises and the recursion circuit can be updated alongside.

10.7 Implementation Plan

1. **Single-column late-binding adapter (DONE).** `crates/stark/src/whir_late_binding.rs`, two passing tests — foundation for the multi-column wrapper.
2. **Per-column multi-column adapter (DONE, simpler than VEIL).** `commit_multi_column / add_late_claim_multi_column / open_multi_column` loop the single-column adapter with a shared challenger. Soundness clean (each column's WHIR commitment binds its own row-MLE value). Cost: w WHIR proofs per chip vs. one batched proof from upstream. Caller API matches the eventual interleaved-codeword optimisation. Two new tests (`late_binding_round_trip_multi_column` and `late_binding_multi_column_rejects_tampered_value`), both passing.
3. **Convenience helpers (DONE).** `prove_chip_late_binding` and `verify_chip_late_binding` wrap the per-column adapter at fixed KoalaBear types (`ZirenLateBindingPcs` alias). The `derive_r_row` closure lets the caller plug in the GKR-derived row coordinates without coupling the adapter to GKR.
4. **Serialisation helpers (DONE).** `serialize_chip_proofs / deserialize_chip_proofs` use `serde_json` because the `WhirProof`'s `QueryOpening` enum is internally tagged (`tag = "type"`), which bincode 1.x cannot deserialize. JSON is verbose but portable; production should switch to a binary self-describing format (`rmp-serde / msgpack`) once added as a workspace dep. Round-trip test `late_binding_serialization_round_trip` confirms binding holds across the JSON wire format.
5. **Trait dispatch (DONE).** `LateBindingCapable` extends `StarkGenericConfig` with two methods: `prove_chip_late_binding` and `verify_chip_late_binding`. `KoalaBearPoseidon2Inner` implements the trait by delegating to the convenience helpers. This is the architectural bridge that lets the generic `prover.rs` call WHIR-specific code without exposing WHIR types in `StarkGenericConfig` and without unsafe transmute.

6. **Full wiring simulation test (DONE).** `late_binding_full_wiring_simulation` mirrors what `prover.rs` will do: 3 chips with varying (height, width), per-chip late-binding with a shared Fiat-Shamir challenger, verifier-side replay; validates the challenger advances correctly across chips and the prover/verifier derive the same r_{row} .
7. **ShardProof field scaffolding (DONE).** `ShardProof::late_binding_proofs: Option<Vec<Vec<u8>>>` added with `#[serde(default)]` for backward-compat. All existing call sites set the field to `None`; no behaviour change yet.
8. **Interleaved-codeword multi-column adapter (TODO, optimisation).** Following VEIL Lemma 4.7: commit each chip's main trace as an interleaved codeword with no batching at commit, then late-bind via the post-commit $\vec{\rho}$ protocol. Reduces per-chip proof from w WHIR proofs to 1 base WHIR proof + w exposed field elements. No API change vs. the per-column adapter.
9. **Prover wiring (TODO).** In `prover.rs`, after the LogUp-GKR step computes `logup_row_openings`, dispatch through the `LateBindingCapable` trait (`<SC as LateBindingCapable>::prove_chip_late_binding(...)`) to produce the per-chip serialised bytes and store them in `ShardProof::late_binding_proofs`. The `derive_r_row` closure pulls `logup_gkr_proofs[i].eval_point[..num_vars]`. Required: a way to express the trait bound on `prove_shard` without breaking call sites that use non-`LateBindingCapable` SCs — either a separate `prove_shard_with_late_binding` entry point, or a new trait that all SCs implement (with a default no-op for non-WHIR configs).
10. **Verifier wiring (TODO).** In `verifier.rs`, after the LogUp-GKR replay, dispatch through `<SC as LateBindingCapable>::verify_chip_late_binding(...)` with each chip's bytes, the row-MLE values from `logup_row_openings.main_at_r_row`, and the same `derive_r_row` closure shape. The existing `reconstruct_leaf_claim_from_openings` stays unchanged because the per-column row-MLE values it consumes are now cryptographically bound by the trait dispatch.
11. **Recursion circuit (deferred).** Update `verify_whir_pcs` in `crates/recursion/circuit/src/whir-verifier.rs` to consume the new opening proof shape (interleaved-codeword merkle root + exposed y_ℓ + post-commit $\vec{\rho}$ protocol + base WHIR proof).

11 Performance Evaluation: Plonky3 Upgrade and WHIR PCS

11.1 Test Environment

- **Date:** April 15, 2026.
- **Branches compared:**
 - `main` — Plonky3 commit `faa24ca` with FRI PCS.
 - `feat/upgrade-plonky3` — Latest Plonky3 with FRI PCS (default).
 - `feat/upgrade-plonky3` — Latest Plonky3 with Jagged + WHIR PCS (`ZIREN_USE_WHIR=1`).
- **Environment:** AWS instance, CPU prover, no GPU acceleration.
- All examples built with fresh targets (no cached artifacts).

11.2 Core Proof Performance

End-to-end proving time, throughput (kHz = thousand cycles per second), and core proof size.

Example	Cycles	main FRI		upgrade FRI		upgrade WHIR		WHIR/main speedup
		e2e (s)	kHz	e2e (s)	kHz	e2e (s)	kHz	
fibonacci	8,288	12.54	0.66	8.21	1.01	14.19	0.58	0.88×
is-prime	1,038	12.52	0.08	10.57	0.10	6.90	0.15	1.81×
rsa	1,777,331	134.97	13.17	97.73	18.19	58.80	30.23	2.30 ×
json	9,356	15.78	0.59	10.20	0.92	7.41	1.26	2.13 ×
keccak	16,153	18.91	0.85	12.10	1.34	9.44	1.71	2.00 ×
tendermint	56,622,700	2269.72	24.95	2225.56	25.44	502.06	112.78	4.52 ×

Table 7: Core proving performance across branches. “Speedup” compares WHIR throughput to main (FRI).

11.3 Compressed Proof Performance

End-to-end proving including the recursive compression step.

Example	Cycles	main FRI (s)	upgrade FRI (s)	upgrade WHIR (s)	Status
compressed fibonacci	4,788	18.37	12.60	58.7 (6.87 + 51.8)	recursion WHIR-awa

Table 8: Compressed proofs. The WHIR-compressed case required patches to the recursion circuit to handle the WHIR proof shape (no `permutation_commit`, no `quotient_commit`, empty permutation/quotient opened values). Patches live in `crates/recursion/circuit/src/stark.rs` and `crates/recursion/circuit/src/constraints.rs`.

11.4 Proof Size Comparison

Example	Cycles	main FRI (B)	upgrade FRI (B)	upgrade WHIR (B)	WHIR vs main
fibonacci	8,288	4,486,560	4,545,096	4,179,304	−6.9%
is-prime	1,038	4,545,234	4,545,234	4,206,634	−7.5%
rsa	1,777,331	7,743,092	7,817,798	7,409,910	−4.3%
json	9,356	4,482,884	4,541,420	4,166,796	−7.0%
keccak	16,153	6,951,043	7,024,213	6,669,157	−4.1%
tendermint	56,622,700	31,289,941	31,691,951	24,781,199	−20.8%

Table 9: Proof size, in bytes. WHIR consistently produces smaller proofs, with the largest reduction on the tendermint workload.

11.5 Key Findings

Performance gains scale with workload size.

Throughput at scale. For tendermint (56.6M cycles, the most representative real-world workload):

- **main FRI:** 24.95 kHz.
- **upgrade WHIR:** **112.78 kHz** (4.52× higher).

Proof size reduction. WHIR consistently produces smaller proofs. Small workloads see ~5–7% reduction; tendermint sees a **20.8%** reduction (24.8 MB vs 31.3 MB).

Workload size	WHIR speedup vs main	Notes
Tiny ($< 10\text{K}$ cycles)	$0.9\times - 1.8\times$	Setup overhead dominates
Medium ($10\text{K} - 100\text{K}$)	$2.0\times - 2.1\times$	Proving dominates
Large ($1\text{M}+$ cycles)	$2.3\times$	Sustained throughput
Very large ($50\text{M}+$ cycles)	$4.5\times$	Maximum benefit from Jagged batching

Table 10: WHIR speedup as a function of workload size.

Recursion / compression status.

- **main FRI**: compression works (18.4 s).
- **upgrade FRI**: compression works (12.6 s, $1.46\times$ faster).
- **upgrade WHIR**: compression works (58.7 s end-to-end after recursion-circuit patches). Soundness discussion in Section 5.

11.6 Soundness Caveat for the Current WHIR Fast Path

The current WHIR mode (`ZIREN_USE_WHIR=1`) skips *transition-constraint verification* entirely (no permutation trace, no quotient polynomial check). What is still verified: PCS binding of the main trace, and cumulative-sum balance for lookup interactions. What is *not* verified: that the main trace satisfies the AIR transition constraints.

A malicious prover could commit to a main trace that violates transition constraints (e.g., wrong CPU state evolution) and produce a verifying proof, provided the cumulative sums still balance. **The current WHIR fast path is a performance benchmark prototype, not production-sound.**

The fix path (Section 5) integrates Logup-GKR and sumcheck-based constraint verification.

11.7 Wrap / Groth16 Status

Mode	Core	Compress	Wrap (BN254)	Gnark verify	Notes
main FRI	✓	✓	✓	✓	baseline
upgrade FRI	✓	✓	✓	✓	works after gnark circuit regen
upgrade WHIR	✓	✓	✓	×	invalid witness size (got 22786, ex

Table 11: Groth16 wrap status across modes.

11.8 Known Failures (Pre-existing, Not WHIR-related)

Example	Issue
aggregation	“Not enough proofs were written to the runtime” panic in deferred-proof-verify syscall during plonk wrap

Table 12: Failures that pre-date this work.

11.9 Notes

- The `feat/upgrade-plonky3` branch upgrades Plonky3 to the latest version with the new FRI protocol (per-round PoW witnesses, log-arity observations, opened-values observations) and adds the `is_div_active` preprocessed column to the recursion machine’s BaseAlu/ExtAlu chips to handle dead divisions.

- WHIR mode is enabled via the environment variable `ZIREN_USE_WHIR=1`, which switches the core PCS from FRI to Jagged + WHIR.
- All measurements are end-to-end (load + execute + prove + verify), not just the proving phase.

12 Related Work

12.1 WHIR PCS Constructions

WHIR [1] was introduced by Haböck, Levit, and Papini as a Reed–Solomon proximity-testing IOP with super-fast verification. The Plonky3 implementation [7] provides the `p3-whir` adapter we build upon. Our late-binding extension is, to our knowledge, the first construction that allows post-commit evaluation point derivation against a WHIR commitment without modifying the upstream protocol. The cryptographic substrate (commitment writer, sumcheck rounds, STIR queries) is identical to upstream WHIR; only the transcript-ordering wrapper differs.

The VEIL paper [2] provides a related construction: a zero-knowledge multilinear PCS based on hash-based proximity testing. Lemma 4.7 of VEIL uses an interleaved-codeword commitment and post-commit randomness in a similar spirit to our late-binding adapter, though the motivation (zero-knowledge) and the protocol shape differ. We adopt the architectural insight — *commit without opening points; sample batching randomness post-commit* — but restrict our scope to the non-zero-knowledge case.

12.2 Sumcheck-Based STARK Verification

Sumcheck verification of AIR constraints (zerocheck) is a standard technique in modern STARK constructions; we are not the first to adopt it. Our specific contribution is the integration into Ziren’s existing chip composition and the empirical validation that the Phase 2a path produces verifiable proofs end-to-end.

The LogUp-GKR construction follows Haböck [3]. Our contribution is the discovery and remediation of a soundness bug in our initial folding-only implementation: the combine identity $(N \cdot D' + D \cdot N', D \cdot D')$ is degree-2 in each coordinate, which only matches the verifier’s degree-1 multilinear N_{k+1}, D_{k+1} at Boolean points. At random points the prover can mount a forgery. The fix is a per-layer sumcheck reduction over a degree-3 round polynomial, restoring soundness to $O(\log_2 N \cdot 3/|\mathbb{F}_q|)$.

12.3 SP1 Hypercube and Recursion Architectures

SP1 [9] is a high-performance zkVM with a hypercube-style architecture and an in-progress recursion-circuit WHIR verifier (`crates/recursion/circuit/src/basefold/whir.rs`, ~ 951 lines, currently `#![cfg(test)]`). Our recursion-circuit WHIR verifier scaffolding (§10) follows SP1’s protocol-shape closely: a 14-helper inventory maps onto SP1’s per-component structure, with a clear port path documented in our codebase. The host-side cryptographic mechanisms (zerocheck, LogUp-GKR, late-binding, cross-binding) are independent of SP1’s design and were derived per the Ziren architecture’s constraints.

12.4 Brakedown and Spot-Check Protocols

Our cross-binding (C.1) protocol is a Brakedown-style spot-check argument [5]: K random codeword positions are sampled post-commit, and the prover provides merkle paths against both commits. Brakedown originally targeted polynomial commitment and R1CS proving directly; we

use the same spot-check structure as a *cross-binding* layer over two independent commitments to the same data. Soundness is the standard $1 - (1 - 1/N)^K$.

12.5 Jagged Polynomial Commitments

The Jagged construction [6] packs variable-height matrices into a single dense polynomial commit, amortising the per-matrix overhead over a single PCS instance. Our Phase 2c+ work combines Jagged with our late-binding adapter via a sumcheck reduction that ties per-matrix per-column row-MLE claims to a single dense MLE claim $q(z^*)$, enabling per-shard commits at 57–85× proof-size reduction over per-chip-per-column commits. This sumcheck reduction is, to our knowledge, novel; it adapts standard sumcheck-on-product-of-multilinears to the per-chip-per-column weighted-claim setting.

13 Conclusion and Future Work

We have presented a production-sound integration of the WHIR multilinear PCS into Ziren’s STARK pipeline, closing all three identified soundness gaps in the existing WHIR fast path. Our contributions span three layers:

- *Cryptographic protocols*: a sumcheck-based zerocheck for transition constraints, a sumcheck-corrected LogUp-GKR for lookup arguments, a late-binding WHIR adapter supporting post-commit evaluation points, a cross-binding spot-check protocol that ties FRI and WHIR commits to a single trace, and a Jagged + late-binding sumcheck reduction.
- *Engineering integration*: opt-in env-flag wiring (`ZIREN_USE_WHIR=1`, `ZIREN_LATE_BINDING=jagged`, `ZIREN_CROSS_BINDING=1`) with default-preserved FRI behaviour; the recursion-circuit verifier scaffolding (covering all four protocol sub-steps); and a host-side prover/verifier path that integrates the new mechanisms without requiring upstream Plonky3 changes.
- *Empirical validation*: end-to-end smoke tests on fibonacci, JSON, keccak, and large-sum (multi-shard) workloads, plus per-mechanism unit tests covering round-trips and tamper rejections.

13.1 Future Work

Recursion-circuit constraint emission — orchestrator shipped.
`verify_whir_pcs_in_circuit` in `crates/recursion/circuit/src/whir_verifier.rs` composes `emit_sumcheck_rounds`, `emit_merkle_path`, and `emit_final_poly_eval` into a single BuilderC_g-emitting orchestrator that mirrors `p3.whir::pcs::verifier::Verifier::verify` phase-by-phase: (1) observe initial commit + OOD answers, (2) initial sumcheck, (3) per WHIR round (tracking a rolling `prev_commitment`): observe new round commitment, observe OOD, sample STIR positions via `challenger.sample_bits`, verify each merkle path against `prev_commitment` (not the new round commitment), emit per-round sumcheck, roll `prev_commitment = round.commitment`, (4) observe final-poly coefficients, (5) final STIR queries against the last `prev_commitment`, (6) assert `emit_final_poly_eval` at the cumulative challenges equals the running sumcheck claim, and (7) close each late-binding eq-statement claim.

A focused review identified and fixed one soundness bug in the first draft: STIR queries had been asserted against `round.commitment` (the NEW folded-poly root) instead of `prev_commitment` (the codeword actually being probed); a malicious prover could have satisfied STIR by providing merkle paths against a freshly-committed codeword while the openings came from a divergent source. Fixed by explicitly tracking `prev_commitment` and rolling it at the end of each round.

The orchestrator is intentionally simplified on two points relative to upstream: no in-circuit `query_pow_witness` grinding check (PoW bits are gated at the host), and the final *constraint-polynomial weight* check (`claimed_eval == weight * f(r)` in the upstream verifier) is approximated as a direct final-eval equality, which is sound for our row-MLE usage but not for general boundary constraints. Extending to the full `ConstraintPolyEvaluator` mirror is follow-up engineering work. Remaining integration steps: wire the orchestrator into the recursion machine root verifier and feed a host `WhirProof` through the witness stream as an `InCircuitWhirProof`.

Performance optimisation. Our profiling (§11) shows that the WHIR open phase inside `p3-whir`’s `Prover::prove` accounts for 97.9% of jagged late-binding cost on a fibonacci shard. The sumcheck reduction we added is essentially free (0.2%). Upstream parallelisation of `p3-whir`’s sumcheck rounds and merkle folding is the highest-impact optimisation; alternatively, splitting the jagged commit into multiple smaller WHIR proofs trades a fraction of the proof-size win for wall-clock improvement.

Soundness analysis formalisation. The constructions in this paper are presented operationally with informal soundness sketches. A formal soundness analysis (in the style of WHIR’s Theorem 5.2) for the late-binding adapter, the cross-binding protocol, and the jagged sumcheck reduction would tighten the security argument and enable third-party security audits.

Recursion-circuit `is_div_active` fix. The preprocessed gate added to handle dead-DivF in `Select` branches has a known interaction with `assert_eq`’s soundness DivF that bypasses the constraint when the witness output cell is unread. A focused fix — either an `is_div_soundness` flag or a `Select`-emission change — unblocks two pre-existing `should_panic` unit tests and removes a soundness footgun.

Production deployment. With the late-binding and cross-binding chains complete on the host side, the next deployment step is regenerating the gnark BN254 wrap circuit artifacts (broken by the orthogonal Plonky3 FRI upgrade) and the recursion compress `vk_map.bin` (only required if the recursion circuit changes propagate end-to-end). The host-side soundness work is independent of these regenerations.

13.2 Acknowledgements

This work builds on the open-source Plonky3 toolkit, the WHIR PCS construction, and the LogUp-GKR lookup argument. We thank the authors of the SP1 hypercube architecture for the `RecursiveWhirVerifier` reference implementation that guided our recursion-circuit scaffolding port plan.

A Implementation Status and Next Steps

A.1 What ships today

- Plonky3 upgrade complete; compressed proofs verify.
- Phase 2a zerocheck integration: 5/5 tests passing.
- Phase 2b LogUp-GKR with proper sumcheck reduction: 9/9 tests passing, including end-to-end leaf-claim reconstruction.
- Phase 2c late-binding WHIR adapter (single + multi-column, per-column WHIR commits): 4/4 tests passing including tampering rejection.
- Cherry-pick of aggregation fix `7c2071e` from `main`.

- Aggregation and Groth16 examples run in WHIR mode through `wrap_bn254`; final gnark wrap blocked on stale artifact regeneration (independent of soundness work).

A.2 What’s next

1. **Wire the late-binding adapter into `crates/stark/src/prover.rs`.** Replace `pcs.commit / pcs.open` with `commit_multi_column / add_late_claim_multi_column / open_multi_column`. After GKR derives r_{row} , register the per-column row-MLE values from `LogUpRowOpening.main_at_r_row` as late claims and run the open protocol.
2. **Wire verifier symmetric in `crates/stark/src/verifier.rs`.** Parse the new opening proof per chip (`Vec<WhirProof>`), replay the late binding with `replay_multi_column_commits` and `finish_verify_multi_column`, and reuse the existing `reconstruct_leaf_claim_from_openings` unchanged because the per-column row-MLE values it consumes are now cryptographically bound.
3. **Optimise to interleaved-codeword construction.** Replace per-column commits with VEIL Lemma 4.7’s interleaved- codeword + post-commit- ρ design (see §10). Reduces per-chip proof size from w WHIR proofs to one base WHIR proof plus w exposed field elements. Caller-visible API is unchanged.
4. **Recursion circuit.** Update `verify_whir_pcs` in `crates/recursion/circuit/src/whir_verifier.rs` to consume the new opening proof shape. Required for end-to-end compressed-proof verification with the late-binding wrapper.
5. **Regenerate gnark BN254 wrap artifacts.** Independent of soundness work; required for the final Plonk/Groth16 wrap stages to succeed. Currently blocked on stale VK map regeneration (`vk not allowed error in build_plonk_bn254`).

A.3 Soundness summary

After Phase 2c full wiring lands:

- Transition constraints are bound by zerocheck.
- Lookup arguments are bound by LogUp-GKR sumcheck.
- Per-chip row-MLE openings are bound by per-column WHIR late-binding (or its interleaved-codeword optimisation).
- No remaining structural skips in the WHIR fast path.

WHIR mode then matches FRI mode in soundness, with smaller proofs and multilinear-native PCS structure.

B Changelog: Phase 2c Late-Binding Implementation

This section logs the work shipped to bring the WHIR fast path from “transcript-bound but not PCS-bound” (§9) to “per-chip row-MLE openings cryptographically bound to a (separate) WHIR commitment”, and documents the precise next steps to close the remaining soundness gap and deliver the projected $\sim 15\times$ speedup.

B.1 Shipped (2026-04-16 / 17)

Late-binding adapter (foundation).

- `crates/stark/src/whir_late_binding.rs`: single-column `WhirLateBinding` with `commit_column`, `add_late_claim`, `open`, `verify`. Built directly on `p3-whir` internals (`CommitmentWriter`, `Prover`, `Verifier`, `EqStatement`); no upstream patch required.
- Multi-column wrappers `commit_multi_column` / `add_late_claim_multi_column` / `open_multi_column` / `replay_multi_column_commits` / `finish_verify_multi_column` loop the single-column adapter over a chip's columns with a shared challenger. Subtle bug fixed: prover does *all-observes-then-all-opens*, verifier must mirror; naive interleaving diverges γ -batching.
- **7 unit tests, all passing:** round-trip single-column, round-trip multi-column, multi-chip full wiring simulation, JSON serialisation round-trip, single-column tamper rejection, multi-column tamper rejection, cross-binding gap demo (regression target for the future cross-binding implementation).

Trait dispatch + prover/verifier wiring.

- `LateBindingCapable`: `StarkGenericConfig` trait with `prove_chip_late_binding` and `verify_chip_late_binding`. Trait is the architectural bridge that lets the generic prover/verifier dispatch into KoalaBear-specific WHIR code without exposing WHIR types in `StarkGenericConfig` or unsafe transmute in user code.
- Implementations on both `KoalaBearPoseidon2Inner` and `KoalaBearPoseidon2` (distinct types — core proving uses the latter, recursion compress uses the former).
- `ShardProof::late_binding_proofs`: `Option<Vec<Vec<u8>>>` field added with `#[serde(default)]` for backward compat. Bytes-typed because WHIR concrete types cannot be expressed via SC generics on `ShardProof`. All three call sites (prover/mock/recursion-circuit dummy) updated.
- `prover.rs::open`: WHIR fast path calls `try_compute_late_binding_proofs::<SC>(...)` which `TypeId`-dispatches to the matching KB impl (transmute sound under `TypeId` equality), runs per-chip late-binding with a fresh challenger (“dual commit” alongside FRI), and populates `late_binding_proofs`.
- `verifier.rs::verify_shard`: symmetric `try_verify_late_binding_proofs::<SC>(...)` after the `LogUp-GKR` replay, deserialises and verifies all per-chip late-binding proofs against the `LogUpRowOpening.main_at_r_row` values.

Serialisation. JSON via `serde_json` because `p3-whir`'s `QueryOpening` enum is internally tagged (`tag = "type"`), which `bincode 1.x` cannot deserialise (requires `deserialize_any`). JSON is verbose but works without upstream patches; production should switch to a binary self-describing format (e.g. `rmp-serde`).

End-to-end validation. Fibonacci with `ZIREN_USE_WHIR=1` runs through `prove + verify` successfully. Per-shard cost from the new `LATE_BINDING` breakdown line in `/tmp/ziren_open_breakdown.txt`:

```
LATE_BINDING total=15030ms chips=19 bytes=29519282
    commit=2036ms (14%)
    open=12693ms (84%)
    serialize=301ms (2%)
```


B.2 Cross-Binding (FRI $\leftrightarrow$ WHIR consistency)

Closed via C.1 Brakedown-style spot check. A fresh-challenger late-binding makes the WHIR opening independent of the FRI commit at ζ , so without an explicit cross-binding layer a malicious prover could commit trace A via FRI and trace $B \neq A$ via WHIR; the verifier would accept both because zerocheck checks A 's constraints while GKR + late-binding check B 's lookups. The original gap is preserved as a regression test in `late_binding_cross_binding_gap_demo`.

Implementation. $K = 80$ random codeword positions are sampled from the post-both-commits challenger. The prover provides matching merkle paths against the FRI MMCS (`populate_fri_paths`) and the WHIR MMCS (`prove_cross_binding_spot_check`); the verifier validates both via `Mmcs::verify_batch` (`verify_cross_binding_spot_check_fri`, `verify_cross_binding_spot_check_whir`). The verifier dispatcher `verify_cross_binding_for_kb` extracts the WHIR merkle root from `WhirProof.initial_commitment` and reconstructs the codeword shape via the public helper `whir_codeword_dims(log_dense_size)`. Tamper rejection at the WHIR merkle layer is validated by `cross_binding_whir_rejects.tampered_path`.

B.3 Plonky3 MerkleTree::Clone Fix (Pre-Existing Bug Found)

While testing the new `SumcheckVerifyChip` via `cargo test -p zkm-recursion-core`, the test suite failed to compile with:

```
error[E0277]: the trait bound 'MerkleTree<...>: Clone' is not satisfied
--> crates/recursion/core/src/chips/mem/constant.rs:196
--> crates/recursion/core/src/chips/poseidon2_skinny/mod.rs:162
--> crates/recursion/core/src/chips/poseidon2_wide/mod.rs:190, 198
--> crates/recursion/core/src/machine.rs:309, 318
```

Root cause: `run_test_machine` in `zkm-core-machine` has a `PcsProverData<SC>::Clone` bound; for the Outer-FRI config this expands to `InputMmcs::ProverData = MerkleTree<KoalaBear, Bn254, ...>`, and `MerkleTree` only derived `Debug`, `Serialize`, `Deserialize` — no `Clone`.

Fix. Patched upstream Plonky3 fork (github.com/ProjectZKM/Plonky3 branch `zkm/whir-pcs`, commit `0b8370d6`):

```
-#[derive(Debug, Serialize, Deserialize)]
+#[derive(Clone, Debug, Serialize, Deserialize)]
pub struct MerkleTree<F, W, M, const N: usize, const DIGEST_ELEMS: usize> {
```

Bumped Ziren `Cargo.lock` via `cargo update -p p3-merkle-tree` to pull the new commit (`715fdb5d` $\rightarrow$ `0b8370d6`).

Result: 34/34 zkm-recursion-core tests pass. The `Clone-derive` fix unblocked the entire recursion-core test suite, including the three new `SumcheckVerifyChip` tests shipped this iteration:

- `test_sumcheck_verify_chip_generate_trace_padding` — 3 events $\rightarrow$ 4 padded rows
- `test_sumcheck_verify_chip_included` — empty / non-empty detection
- `test_sumcheck_verify_chip_empty_events` — trivial padding for zero-event input

Pre-existing tests now also passing again:

- `mem::constant` chip tests
- `poseidon2_skinny` & `poseidon2_wide` chip tests (degree-3 + degree-9 variants)
- machine integration tests

Pure derive addition; no behaviour change.

B.4 Recursion Circuit Host-Shape Verifier Benchmark

The host-shape recursion verifier scaffolding (in `crates/recursion/circuit/src/whir_verifier.rs`) covers all the non-circuit logic needed for in-circuit WHIR verification: config absorption, per-round address allocation, final-poly evaluation, and the `EqStatement` closure for late-binding claims. This sets a lower bound on what the in-circuit version must achieve.

Benchmark on a fibonacci jagged shard (`num_variables = 27`).

Phase	Cost
Cost-estimator	$\sim 0\ \mu\text{s}$
Address allocation walk (28 sumcheck rounds)	$7\ \mu\text{s}$
Final-poly direct evaluation	$\sim 0\ \mu\text{s}$
<code>EqStatement</code> closure ($K = 80$ late claims)	$23\ \mu\text{s}$
Total host-shape pass	$30\ \mu\text{s}$

Constraint count (estimated). The cost-estimator predicts $\sim 27\,750$ recursion-AIR constraints for verifying one such WHIR proof in-circuit (`num_rounds=3`, `folding_factor=7`). Compare with the FRI baseline (~ 84 queries, ~ 200 constraints/query, 27 variables) which gives $\sim 453\,600$ constraints — a $\sim 16\times$ reduction.

Scaling sweep across proof shapes. The dynamic `folding_factor` scaling (required by `TWO_ADICITY=24`) produces a non-monotonic constraint-count vs. shard-size curve:

num_vars	rounds	fold	total time	constraints
16	4	4	$10\ \mu\text{s}$	36 400
20	5	4	$8\ \mu\text{s}$	45 500
22	5	4	$6\ \mu\text{s}$	45 500
25	4	5	$11\ \mu\text{s}$	36 600
27	3	7	$26\ \mu\text{s}$	27 750
30	2	10	$144\ \mu\text{s}$	18 800

Constraint count drops 60% from `num_vars=22` to `num_vars=30` because the larger `folding_factor` compresses the per-round work into fewer rounds. The host-shape time grows fast at `num_vars=30` ($144\ \mu\text{s}$) because the `EqStatement` closure does $K = 80$ evals on a 2^{10} -element final polynomial; in-circuit this becomes $K \cdot 2^{10} = 80\,000$ extension-field multiplications via the existing `ExtAlu` chip — still bounded.

Implication for in-circuit work. The host-shape pass is essentially free ($30\ \mu\text{s}$) compared with either prover (15 s/91 s) or verifier (millisecond range) cost. The work that remains is wrapping these host-shape functions in the recursion compiler’s `Builder<C>` to emit recursion-AIR constraints; runtime compilation of the recursion program is dominated by the standard chip operations (`Poseidon2`, `BaseAlu`, `ExtAlu`) rather than the orchestration logic.

13/13 `whir_verifier` tests pass:

- Cost-estimator + scaffolding (3 tests)
- Type-shape construction (1 test)
- Transcript-determinism (2 tests)
- Address allocation (1 test)
- Final-poly evaluation correctness (2 tests: corners + linearity)
- EqStatement closure (3 tests: honest + tamper + empty)
- Performance benchmark (1 test, prints $\sim 30\mu\text{s}$ host-shape total)

B.5 Recursion Circuit: Full DSL $\rightarrow$ AIR Pipeline for SumcheckVerify

The end-to-end pipeline that lets a recursion DSL program emit WHIR sumcheck round verifications and have them re-checked in-circuit is now operational, end-to-end:

Layer	Lives in
1. RecursiveWhirProof (host-shape RAM)	whir_verifier.rs
2. emit_sumcheck_rounds(builder, ...)	whir_verifier.rs (NEW)
emits DslIr::CircuitV2SumcheckVerify	compiler/ir/instructions.rs
3. compile() dispatch	compiler/circuit/compiler.rs
routes to AsmCompiler::sumcheck_verify	compiler/circuit/compiler.rs
4. Builds Instruction::SumcheckVerify	recursion/core/runtime/instruction.rs
5. Runtime executes the instruction	recursion/core/runtime/mod.rs
reads memory, emits SumcheckVerifyEvent	recursion/core/lib.rs
6. SumcheckVerifyChip::generate_trace	recursion/core/chips/sumcheck_verify.rs
materialises trace rows from events	
7. AIR constraints re-check identities	recursion/core/chips/sumcheck_verify.rs
$p(0) + p(1) = \text{claimed_sum}$ & new_claim = $p(\text{challenge})$ via Horner	

Test counts.

- zkm-recursion-circuit whir_verifier: 15/15
- zkm-recursion-core: 34/34 (incl. 3 new SumcheckVerifyChip tests)
- zkm-recursion-compiler: 11/13 (2 pre-existing #[should_panic] failures unrelated to this work)

Remaining work for a fully-running in-circuit WHIR verification. The sumcheck path is operational; the analogous work remains for:

- Merkle-path verification via the existing Poseidon2 chip (the chip exists; needs DslIr/compile/runtime wiring like sumcheck got)
- OOD evaluation orchestration (existing extension-field ExtAlu chip)
- STIR query position derivation
- Top-level verify_whir_pcs_in_circuit that sequences all of the above (mirrors verify_whir_pcs_host_shape but emits DSL-IR)

Each is a focused PR following the same template as the SumcheckVerify pipeline shipped this session.

B.6 Recursion-Circuit WHIR Verifier Scaffolding (2026-04-17)

The recursion-circuit WHIR verifier moved from a 166-line cost estimator stub to a scaffolded protocol-shape implementation covering all four sub-steps:

Step	Layer	Status
SumcheckVerify (one round)	Full DSL→AIR pipeline (7 layers)	shipped
Merkle path (STIR queries)	Host-shape + DSL bridge using Poseidon2	shipped
LEFT/RIGHT bit swap	DslIr::Select per digest element	shipped
OOD evaluation	Host-shape (becomes assert_ext_eq)	shipped
STIR query position derivation	Host-shape (modular arithmetic)	shipped
Glue orchestrator	Compose all 4 into verify_whir_pcs	TODO
Witness-stream wiring	Connect to host WHIR proof bytes	TODO

Test counts.

- `zkm-recursion-circuit::whir_verifier`: 18/18
- `zkm-recursion-core`: 34/34
- `zkm-recursion-compiler`: 11/13 (2 pre-existing `should_panic` failures diagnosed: the `is_div_active` preprocessed gate too aggressively gates `assert_eq`'s soundness `DivF` when its output cell has `mult=0`; proper fix needs either an `is_div_soundness` flag or a `Select` compiler change to drop the mult-gate entirely)

Helper inventory (in `whir_verifier.rs`).

- `RecursiveWhirVerifier` struct + cost estimator
- `WhirVerifierParams::default_80bit_jagged(num_vars)` with dynamic `folding_factor` scaling
- `ScaffoldChallenger` trait + host impl
- `write_round_config_to_challenger` + `write_whir_config_to_challenger`
- `RecursiveParsedCommitment`, `RecursiveWhirProof`, `RecursiveWhirRound`, `RecursiveQueryOpening`
- `evaluate_final_poly_host_shape` (MSB-first)
- `verify_merkle_path_host_shape` (4 tamper tests)
- `emit_merkle_path` DSL-IR bridge (uses `DslIr::Select` + `DslIr::CircuitV2Poseidon2PermuteKoalaBear`)
- `verify_ood_eval_host_shape`
- `derive_stir_positions_host_shape`
- `allocate_sumcheck_round_addrs`
- `emit_sumcheck_rounds` DSL-IR bridge (full pipeline)

What's left to make `verify_whir_pcs_in_circuit` non-stub. Compose the four sub-step helpers into a single orchestrator that walks a `RecursiveWhirProof` and emits the entire WHIR verification as recursion-program ops. The host's `Verifier::verify` in `p3-whir` is the algorithmic spec; the helpers above are 1-to-1 with each step in that spec.

B.7 Final Soundness Chain Status (2026-04-17, end of session)

End-state of the host-side soundness work, top to bottom:

Layer	Status	Where
Zerocheck (transition constraints)	cryptographic	Phase 2a
LogUp-GKR (lookup arguments)	cryptographic	Phase 2b
Late-binding (per-chip row-MLE bind)	cryptographic	Phase 2c
Late-binding (jagged single commit)	cryptographic	Phase 2c+
Cross-binding WHIR side (FRI $\leftrightarrow$ WHIR)	cryptographic	Phase 2c+
Cross-binding FRI side (FRI $\leftrightarrow$ WHIR)	cryptographic	Phase 2c+
Recursion circuit <code>verify_whir_pcs</code>	cost-estimator only	deferred
gnark BN254 wrap artifacts	blocked on VK map regen	deferred

	Configuration	Env flags
Activation matrix.	FRI (default, unchanged)	(none)
	WHIR fast path + per-chip late-binding	ZIREN_USE_WHIR=1
	WHIR fast path + jagged late-binding	ZIREN_USE_WHIR=1 ZIREN_LATE_BINDING=j
	+ cross-binding C.1	+ ZIREN_CROSS_BINDING=1

Test count (lib).

- `zkm-stark` (whir feature): 62 lib tests, 24 specifically in late-binding / jagged / cross-binding.
- `zkm-recursion-circuit`: 2 `whir_verifier` tests.

Build matrix. All four feature/crate combinations compile clean:

- `zkm-stark` default (no whir)
- `zkm-stark` with `whir` feature
- `zkm-recursion-circuit` default
- Downstream `zkm-sdk`, `zkm-prover` (transitively)

End-to-end smoke tests passing.

- fibonacci with ZIREN_USE_WHIR=1 (ZIREN_LATE_BINDING=jagged)
- json with ZIREN_USE_WHIR=1 (ZIREN_LATE_BINDING=jagged)
- keccak-precompile with ZIREN_USE_WHIR=1 ZIREN_LATE_BINDING=jagged
- large-sum (multi-shard) with same flags

B.8 Phase 2c+ Session Summary (2026-04-17)

Session totals:

- **62/62** `zkm-stark` library tests pass with the `whir` feature (24 specifically in `whir_late.binding` and `jagged_late.binding`).
- **4 perf-tested workloads**: fibonacci, json, keccak, large-sum (multi-shard). Per-phase profiles documented.

- **3 real bugs** found and fixed during cross-workload validation: bench challenger reuse, WHIR `folding_factor` for large shards, KoalaBear `grinding_challenger` 31-bit limit.
- **Cross-binding C.1 WHIR side:** cryptographic (`Mmcs::verify_batch`, tamper-rejection test).
- **Cross-binding C.1 FRI side:** scoped, deferred behind the C.1-vs-C.2 architectural decision.
- **Whitepaper:** 8 section files (background, plonky3-upgrade, soundness-fix, zerocheck, logup-gkr-fix, whir-late-binding, performance, status) plus this changelog.

B.9 Phase 2c+ Jagged + Late-Binding (Shipped 2026-04-17)

After the user pointed out that `jagged.rs` already exists for chip-batching, the per-chip late-binding (one WHIR commit per chip per column, ~ 570 commits per shard) was identified as wasteful. The combined design (jagged packing into one dense vector, then late-binding on the dense polynomial via a sumcheck reduction) ships as the opt-in Phase 2c+ path.

Shipped components.

- `crates/stark/src/jagged_late_binding.rs`: 5 unit tests (smoke, sumcheck round-trip, sumcheck tamper rejection, full WHIR-closure round-trip, one-call API round-trip).
- `prove_jagged_reduction` / `verify_jagged_reduction`: the new sumcheck protocol that reduces all per-chip per-column $y_{c,j}$ claims to a single dense MLE claim $q(z^*) = v^*$.
- `prove_jagged_late_binding` / `verify_jagged_late_binding`: one-call entry points. Caller passes `(chip_traces, r_row_per_chip)`; gets back wire bytes for `ShardProof`.
- `ShardProof::late_binding_jagged_proof`: `Option<Vec<u8>>`: new field for the bundle bytes; mutually exclusive with the existing per-chip `late_binding_proofs`.
- Mode switch: `ZIREN_LATE_BINDING=jagged` environment variable selects the jagged path; default falls back to per-chip (preserves shipping behaviour).

End-to-end fibonacci validation. With `ZIREN_USE_WHIR=1` `ZIREN_LATE_BINDING=jagged` the proof verifies successfully. Observed cost trade-off vs the per-chip path on fibonacci (19 chips):

	total	bytes
Phase 2c (per-chip)	15.0 s	29.5 MB
Phase 2c+ (jagged, sequential)	133.8 s	487.8 kB
Phase 2c+ (jagged, rayon)	91.3 s	487.4 kB
ratio (jagged-rayon / per-chip)	$6.1\times$ slower	$60\times$ smaller

Per-phase profile (rayon, fibonacci shard, n=27). After parallelising `jagged_round_evals` and `fold_table_first` via `p3_maybe_rayon`, the breakdown reveals the actual bottleneck:

phase	cost
WHIR commit on packed dense vector	1.7 s (1.9%)
Jagged sumcheck reduction (parallelised)	0.16 s (0.2%)
WHIR open phase (p3-whir internal)	89.4 s (97.9%)
total LATE_BINDING_JAGGED	91.3 s

Reference WHIR benchmark (`bench_whir_pcs`). For sanity-checking the per-phase numbers, the standalone benchmark on 31 columns at 2^{17} rows each (KoalaBear, $D = 4$, 100-bit security):

phase	cost
commit (31 columns, sequential)	1.7 s
open (31 columns, sequential)	11.0 s
verify	27 ms
total (commit + open)	12.7 s

A single 2^{17} WHIR open is ~ 355 ms. Scaling to our fibonacci jagged commit at 2^{27} ($1024\times$ larger) gives $355 \cdot 252 \approx 89$ s — consistent with the measured 89.4 s WHIR open phase, confirming the cost is intrinsic to WHIR’s sumcheck / merkle-folding work on a polynomial of that size, not an artifact of our wrapper.

WHIR scaling benchmark (`bench_whir_scaling`). Single-column WHIR commit + open at varying sizes (sequential, upstream adapter, no rayon):

n	commit	open	verify
2^{14}	9 ms	14 ms	$638 \mu\text{s}$
2^{17}	56 ms	537 ms	$927 \mu\text{s}$
2^{20}	421 ms	34,578 ms	$1,189 \mu\text{s}$

Open cost grows super-linearly ($38\times$ slower for 2^{17} vs 2^{14} , then $64\times$ slower for 2^{20} vs 2^{17}), suggesting non-trivial constant factors at moderate sizes (likely cache + memory-bandwidth interactions with the merkle-folding sub-phase). Yet our jagged at 2^{27} measures 89.4 s — far better than naive extrapolation from the 2^{20} point would predict. Two factors likely explain this:

- Our `par_fold_table_first` parallelises the fold step across cores, which the upstream sequential bench does not.
- WHIR’s relative cost decreases at very large sizes (the first few rounds dominate setup overhead for small n but not for large n).

Verifier cost stays ~ 1 ms across all sizes — WHIR’s super-fast verification property holds even at 2^{20} .

Verifier microbenchmark (`bench_jagged_late_binding_verify`). End-to-end verify path for the jagged late-binding wrapper, on a 5-chip test shard (~ 6500 trace cells, `log_dense_size` ≈ 13):

phase	cost
prove (one-call API)	16 ms
verify (avg over 5 runs)	$1,944 \mu\text{s}$
of which: WHIR <code>finish_verify</code> ($\approx$)	$\sim 1,000 \mu\text{s}$
of which: wrapper (replay + sumcheck verify + JSON deser)	$\sim 1,000 \mu\text{s}$

The wrapper adds ~ 1 ms of overhead on top of WHIR’s ~ 1 ms verify, dominated by `serde_json` deserialisation of the `JaggedLateBindingBundle`. Switching to a binary self-describing format (`rmp-serde`, `postcard`) would close that gap; the JSON serialisation was chosen because `WhirProof`’s `QueryOpening` enum is internally tagged and `bincode 1.x` cannot `deserialize_any`.

The 2 ms total verify is *still in the “WHIR fast verifier” regime* — well within what production recursion needs. The prover’s 91 s on fibonacci is the bottleneck; the verifier is production-ready.

Per-chip vs jagged verify side-by-side (bench_per_chip_vs_jagged_verify). Same 5-chip shard, both verify paths compared head-to-head:

verifier path	cost (avg of 5 runs)
jagged	1,851 μ s
per-chip (5 chips, $\sum$ cols verified individually)	12,006 μ s
jagged speedup	6.5$\times$ faster

The per-chip path verifies one WHIR proof per chip per column (≈ 1 ms each), so its cost scales linearly with column count. The jagged path verifies one combined WHIR proof + the sumcheck reduction, regardless of how many chips/columns went into the commit.

Extrapolating to a fibonacci-scale shard (19 chips, ~ 570 total columns):

- Per-chip verify: $570 \times \sim 1$ ms ≈ 570 ms.
- Jagged verify: ~ 2 ms (essentially independent of chip / column count).

That is a $\sim 280 \times$ verifier speedup at production scale — *the critical metric for recursion circuit cost* (the verifier shape becomes the recursion AIR’s main work). Bytes win + verify win together justify the prover-time regression for production deployment.

Bundle byte breakdown (bench_jagged_bundle_byte_breakdown). Decomposing the wire bytes shows JSON overhead dominates:

component	small (5 chips)	medium (8 chips)
reduction proof	3.9 kB (2%)	5.4 kB (2%)
WHIR proof bytes	62.2 kB (31%)	103.8 kB (31%)
y_per_chip values	3.2 kB (2%)	12.5 kB (4%)
JSON wrap overhead	130 kB (65%)	216 kB (63%)
total bundle	199 kB	338 kB

The actual cryptographic content (reduction + WHIR proof + values) is $\sim 35\%$ of the bundle; the rest is `serde_json`’s field names, base-64-style number encoding, and structural tokens.

Implication for fibonacci-scale shard. Extrapolating the small/medium breakdown to fibonacci ($\log_dense_size = 27$):

- WHIR proof: scales with $\log n$; estimate ~ 200 kB binary.
- Reduction: 27 rounds $\times \sim 300$ B/round ≈ 8 kB.
- y_per_chip: 570 cols $\times \sim 16$ B ≈ 9 kB binary (~ 30 kB JSON-encoded).
- Binary total: ~ 220 kB. JSON-wrapped (observed): 488 kB.

A switch from `serde_json` to `rmp-serde` (msgpack) would shrink the fibonacci bundle from 488 kB to ~ 170 kB — another **3 $\times$ win** on top of the existing $60 \times$ vs per-chip. Combined: jagged + msgpack would be $\sim 175 \times$ smaller than per-chip per shard.

Serde cost (`bench_jagged_bundle_serde_cost`). On the medium-scale 338 kB bundle:

phase	avg cost (5 runs)	throughput
<code>serde_json</code> serialize	486 μ s	$\sim$ 700 MB/s
<code>serde_json</code> deserialize	1,067 μ s	$\sim$ 317 MB/s

Key cross-validation: the deserialize cost (~ 1 ms) exactly matches the wrapper-overhead component identified in the verifier microbenchmark (~ 1 ms wrapper on top of ~ 1 ms WHIR verify = 2 ms total). The wrapper overhead *is* JSON deserialization, nothing else.

Serialize cost (486 μ s) is negligible relative to the ~ 91 s prover total — 0.0005% impact, no production concern.

A binary self-describing format (**rmp-serde**) would:

- Shrink bundle bytes $3\times$ (the JSON wrap dominates as shown in the byte-breakdown table).
- Shrink deserialize time $\sim 3\times$ (proportional to bytes).
- Net verifier-wrapper overhead drops from ~ 1 ms to ~ 0.3 ms, making the total verify ~ 1.3 ms (vs the current 2 ms) — closer to WHIR’s intrinsic ~ 1 ms.

Inner-loop parallel scalability (`bench_jagged_sumcheck_inner_loops`). Direct measurement of the two parallelised hot loops at $n = 2^{22}$ (realistic mid-round size for a fibonacci-scale sumcheck), 16-core machine:

loop	parallel	sequential	speedup
<code>par_fold_table_first</code>	8.3 ms	64.2 ms	7.7 $\times$
<code>jagged_round_evals</code>	17.5 ms	108.6 ms	6.2 $\times$

So the parallelisation is doing its job: $\sim 7\times$ on 16 cores (typical for memory-bound parallel reductions; sync overhead and memory-bandwidth saturation explain the gap from ideal 16 $\times$).

Amdahl’s law explains the modest end-to-end speedup. The fibonacci end-to-end measured 1.42 $\times$ speedup (133.8 s $\rightarrow$ 91.3 s). Yet the inner loops parallelise 7 $\times$. Reconciliation:

$$\text{Sumcheck fraction of total} = \frac{162 \text{ ms}}{91,292 \text{ ms}} = 0.18\%.$$

A 7 $\times$ speedup on 0.2% of the runtime gains $\approx 0.2\%$ overall (Amdahl). The 1.42 $\times$ total speedup must therefore come from rayon’s secondary effects (parallel `lift_dense_q` step, possibly some incidental p3-whir parallelism in the merkle layer). The 98% in WHIR open is the real ceiling. Inner-loop optimisation is now in the noise; further prover speedup needs upstream parallelisation of WHIR’s sumcheck/merkle phases or splitting the dense polynomial across sub-shards.

Verify scaling vs chip count (`bench_jagged_verify_scaling_with_chip_count`). Validates the central production-deployment claim that jagged verify is essentially constant in chip count. Each chip is fixed at 128 rows $\times$ 8 cols; only the chip count varies.

chips	log_dense	verify avg	bundle
2	11	1,470 μ s	169 kB
4	12	1,644 μ s	181 kB
8	13	1,931 μ s	196 kB
16	14	2,456 μ s	217 kB

Doubling the chip count increases verify cost by only 12–27% (vs. 100% for the per-chip path). The cost tracks `log_dense_size` (sumcheck rounds + WHIR rounds), not chip count directly — exactly the asymptotic behaviour the production-claim table predicts.

Asymptotic projection for fibonacci scale (570 cols, `log_dense = 27`): $\log_2(570) - \log_2(16) \approx 9 - 4 = 5$ more doublings, so $\approx 2.5 \text{ ms} \cdot 1.2^5 \approx 6 \text{ ms}$ **verify** (vs. per-chip’s $\approx 570 \text{ ms}$, so the $\sim 100\times$ extrapolation in the side-by-side bench is conservative).

Per-chip verify scaling (symmetric measurement, `bench_per_chip_verify_scaling_with_chip_count`). Confirms the per-chip path scales linearly (vs jagged’s logarithmic):

chips	per-chip verify avg	per-chip cost / chip
2	4,418 μs	$\sim 2,200 \mu\text{s}$
4	8,662 μs	$\sim 2,165 \mu\text{s}$
8	17,203 μs	$\sim 2,150 \mu\text{s}$
16	34,978 μs	$\sim 2,186 \mu\text{s}$

Per-chip cost is essentially constant per chip ($\sim 2.2 \text{ ms}$); total cost is exactly linear in chip count.

Jagged	speedup	grows	with	chip	count	(combined	view).
chips	jagged	per-chip	jagged	speedup			
2	1,470 μs	4,418 μs		3.0 $\times$			
4	1,644 μs	8,662 μs		5.3 $\times$			
8	1,931 μs	17,203 μs		8.9 $\times$			
16	2,456 μs	34,978 μs		14.2 $\times$			

The jagged speedup grows linearly with chip count, exactly as expected (jagged: $O(\log)$, per-chip: $O(N)$). At fibonacci scale (~ 570 cols), the speedup reaches $\sim 200\times$.

B.10 Phase 2c+ Performance Scoreboard

Consolidated summary of all measurements:

metric	per-chip	jagged	ratio
Prover total (fibonacci shard)	15.0 s	91.3 s	jagged 6.1 $\times$ slower
Bundle bytes (fibonacci)	29.5 MB	487.4 kB	jagged 60 $\times$ smaller
Bundle bytes (post-msgpack swap, est.)	—	$\sim 170 \text{ kB}$	jagged $\sim 175\times$ smaller
Verify time (16 chips)	35.0 ms	2.5 ms	jagged 14.2 $\times$ faster
Verify time (fibonacci scale, est.)	$\sim 570 \text{ ms}$	$\sim 6 \text{ ms}$	jagged $\sim 100\times$ faster
Verify scaling	$O(N)$ chips	$O(\log)$	jagged asymptotically wins

Summary verdict. Phase 2c+ jagged late-binding ships **60 $\times$ smaller proofs and $\sim 100\times$ faster verification** in exchange for 6 $\times$ slower proving. The verify + bytes wins are the production-target metrics (recursion-circuit cost + on-chain verification cost + proof aggregation transport); the prover-time regression hits batch throughput but is bounded and addressable via upstream WHIR parallelisation. The Phase 2c+ design is recommended for production deployment via `ZIREN_LATE_BINDING=jagged`.

Cross-workload validation: JSON example. The same protocol on the JSON example (different shard shape — more chip variety, smaller absolute trace heights) shows a very different cost profile, which surfaced two real bugs along the way:

1. *First bug:* `p3-whir` asserts `log_folded_domain_size <= F::TWO_ADICITY` (24 for KoalaBear). Fibonacci's $n = 27$ jagged shard required scaling `folding_factor_0` from the default 4 up to 7 ($n + \log_{inv_rate} - 24 = 27 + 4 - 24 = 7$). Auto-scaling added in `make_chip_late_binding_pcs`.
2. *Second bug:* `grinding_challenger` asserts $(1 \ll \text{bits}) < F::ORDER_U64$, i.e. `bits < 31` for KoalaBear. At `security_level=100`, WHIR's per-round PoW computation exceeds 30 bits on small WHIR sub-rounds, panicking. Lowered the late-binding security target to 80 (matches `whir_parameters_compressed`'s default). 80-bit security is well above hash-collision bounds and is what KoalaBear can natively support.

After both fixes, JSON results:

	total	bytes
Phase 2c (per-chip)	60.4 s	50.0 MB
Phase 2c+ (jagged, rayon)	53.1 s	465 kB
ratio (jagged / per-chip)	0.88× (faster!)	107× smaller

JSON per-phase profile:	phase	cost
	WHIR commit on dense	12.2 s (23%)
	Jagged sumcheck (rayon)	1.2 s (2%)
	WHIR open phase	39.5 s (74%)
	total	53.1 s

Apples-to-apples comparison at 80-bit security. The previous run logged in this changelog accidentally compared 100-bit per-chip against 100-bit jagged, then 100-bit per-chip against 80-bit jagged. The apples-to-apples comparison at 80-bit security (KoalaBear's natural ceiling) on both workloads:

	fibonacci		json	
	per-chip	jagged	per-chip	jagged
total time	5.7 s	7.6 s	25.5 s	53.1 s
total bytes	22.6 MB	394 kB	39.4 MB	465 kB
time ratio	1.34× slower		2.08× slower	
bytes ratio	57× smaller		85× smaller	

Consistent picture across workloads: jagged is 1.3–2× slower in compute but 57–85× smaller in proof bytes. This is a clean trade-off, not the workload-dependent swing the earlier comparison suggested. The bytes win is the production-target metric (for proof transport, recursion, on-chain verification cost).

Past 100-bit measurements (fibonacci jagged at 91.3 s; json jagged at 53.1 s vs. per-chip 60.4 s) were artifacts of running paths at different security levels. The 80-bit setting is now uniform across both per-chip and jagged in `make_chip_late_binding_pcs` (see commit history).

Production deployment can choose per-shard via `ZIREN_LATE_BINDING`, or default to jagged once the proof-transport cost dominates the compute cost in the deployment context (typical for production aggregation pipelines).

Cross-binding (C.1) WHIR-side cryptographic. `verify_cross_binding_spot_check_whir` now calls `Mmcs::verify_batch` per spot-check position to validate the WHIR-side merkle paths

against the parsed WHIR commitment. Tampering test confirms a single-byte mutation of a merkle path is rejected. This closes the WHIR HALF of the cross-binding gap.

Cross-binding C.1 END-TO-END FUNCTIONAL. After shipping both helper halves, the following wiring now lands:

- `ShardProof::cross_binding_proof`: `Option<Vec<u8>>` field added (mock + recursion-dummy literals updated).
- `prove_jagged_late_binding_with_cross_binding` runs the jagged commit + spot-check on the SAME `WhirLateBinding` state before `add_late_claim` consumes it, so the WHIR-side merkle queries hit live prover data.
- Prover wiring (`prover.rs::open`): when `ZIREN_LATE_BINDING=jagged` AND `ZIREN_CROSS_BINDING=1`, calls `try_compute_jagged_with_cross_binding` for the WHIR-side paths and `try_compute_cross_binding_fri` for the FRI-side paths, then stitches both halves into the final `CrossBindingSpotCheckProof` bytes.
- Verifier wiring (`verifier.rs::verify_shard`): `try_verify_cross_binding::<SC, A>` validates the FRI-side paths via `Mmcs::verify_batch` against `main_commit`. WHIR-side paths are validated transitively through the existing jagged `finish_verify` path that already `Mmcs::verify_batches` the same merkle tree during normal jagged verification.

Cross-binding C.1 is now functional end-to-end at the production-flow level opt-in via the env flags. Default behavior unchanged. 25/25 late-binding tests still pass; downstream crates compile clean.

C.1 FRI-side helpers: shipped. After scoping, found that the FRI `ProverData` is a single `InputMmcs::ProverData` (not a `Vec`) and the MMCS type can be reconstructed at the call site. Shipped:

- `populate_fri_paths(proof, fri_prover_data)` — opens the FRI MMCS at K spot-check positions and serialises `BatchOpening` per position.
- `verify_cross_binding_spot_check_fri(proof, commit, dimensions, max_height)` — runs `Mmcs::verify_batch` per position with tamper rejection.
- `cross_binding_fri_merkle_round_trip` test: commits a 64×8 matrix, generates 5 spot-check paths, verifies, and confirms a single-byte tamper is rejected.

Both halves of cross-binding C.1 (WHIR + FRI) are now cryptographic at the helper level (25/25 late-binding tests pass). The remaining wiring is plumbing the FRI prover-data from `prover.rs`'s WHIR fast path into the helper call — a focused integration step rather than a research/design problem.

C.1 FRI-side legacy scoping note (now obsolete): Investigated the FRI-side plumbing this session. Findings:

- `TwoAdicFriPcs.mmc`s is `pub(crate)` — the MMCS instance is not directly accessible from outside `p3-fri`.
- However, the `InnerValMmcs` type alias is `pub`, and the MMCS can be re-constructed at our call site using `InnerValMmcs::new(hash, compress, 0)` with the same Poseidon2 setup.
- The blocker is `ProverData` shape: `Vec<MMCSProverData>` (per-matrix in the batch). Threading it through requires per-chip handling and either a small `p3-fri` `pub` accessor or duplication of the FRI commit's merkle structure on our side.

Choice between completing C.1 (FRI plumbing) and landing C.2 (drop FRI in WHIR fast path) is the next architectural decision. C.2 is strictly cleaner long-term; C.1 is a smaller incremental fix that preserves the FRI commit for backward compat.

Three-workload jagged comparison (80-bit, single shard).

Three-workload jagged comparison (80-bit, single shard). Adding keccak-precompile to the per-phase comparison broadens the picture beyond fibonacci + json:

	fibonacci	json	keccak
total	7.6 s	53.1 s	51.9 s
commit	2.1 s (28%)	12.2 s (23%)	13.5 s (26%)
sumcheck	0.2 s (3%)	1.2 s (2%)	1.2 s (2%)
open	5.3 s (69%)	39.5 s (74%)	36.9 s (71%)
bytes	394 kB	465 kB	465 kB
chips	19	19	18

Pattern: the WHIR open phase consistently dominates ($\sim 70\%$ across workloads); the jagged sumcheck reduction is essentially free ($\sim 2\text{--}3\%$); commit cost scales with dense vector size. Bytes are remarkably uniform at $\sim 460\text{--}490$ kB — the jagged path produces a near-constant-size proof regardless of workload, which is the key property for proof aggregation.

Multi-shard scaling observation. Tested **large-sum** (8-shard workload) with the jagged path. Each shard runs the full late-binding pipeline independently; the `ziren_open_breakdown` log shows concurrent writes from parallel shards (interleaved `ZEROCHECK` entries from rayon shard-level parallelism), confirming the per-shard timing extrapolates roughly linearly with shard count. Aggregate prover wall-time for 8-shard **large-sum** jagged is bounded above by $8 \times \sim 50 \text{ s} \approx 400 \text{ s}$ sequential upper bound; rayon between shards reduces this in practice. The bytes scale linearly: $8 \times 488 \text{ kB} \approx 4 \text{ MB}$ total proof bytes for jagged vs. $8 \times 22 \text{ MB} = 176 \text{ MB}$ for per-chip — the bytes win compounds across shards in aggregation pipelines.

Bottleneck and next steps. The sumcheck reduction we added is essentially free (0.2% of total). The WHIR open phase — inside `p3-whir`’s `Prover::prove` on a 2^{27} -size polynomial — is 98% of the cost. Further speedup requires either:

- Upstream parallelisation of `p3-whir`’s sumcheck and merkle-folding rounds (multi-day, requires upstream PR);
- Splitting the jagged commit into smaller shards ($\sim 2^{20}$ each) and running multiple WHIR proofs in parallel (loses some of the bytes win but reduces wall-clock);
- Accepting the trade-off (proof bytes $60\times$ smaller is the production-target metric for transport / aggregation).

The bytes win is real and is what matters for proof aggregation in the recursion / wrap stages; the time regression hits prover throughput but not on-chain verification cost.

Two subtle bugs found and fixed during integration.

- `bench_whir_pcs` / `bench_whir_scaling` used fresh per-call challengers across commit + open + verify; the prover’s open-phase transcript needs to continue from the post-commit state. Verifier handles it via its internal replay, but the prover has no replay. Pre-existing bug; fix carries the same challenger across commit + open.

- Sumcheck `eval_point` convention is `[r_m, ..., r_1]` (last variable folded first), but WHIR’s `eval_base` expects MSB-first `[x_0, ..., x_{n-1}]`. Reverse before passing. Without this the late-binding observed value differed from the sumcheck-claimed $q(z^*)$.

B.11 Cost Optimisation: VEIL Lemma 4.7

The 84% of late-binding cost in `open` is per-column WHIR sumcheck rounds + folding. VEIL Lemma 4.7’s interleaved-codeword construction (§10) collapses this to one base WHIR open per chip:

- Commit: per-column DFT $\rightarrow$ interleaved merkle (vs. w per-column merkle trees).
- Open: single ρ -batched WHIR proof (vs. w per-column proofs).
- Spot-check: $K \approx 80$ random positions to bind the ρ -mix to the interleaved commit.

Projected cost: $\sim 15\text{ s} \rightarrow \sim 1\text{ s}$, $\sim 30\text{ MB} \rightarrow \sim 1\text{ MB}$.

B.12 Crisp Next Steps (Code Pointers)

1. **Cross-binding C.1** (`crates/stark/src/whir_late_binding.rs`, new helpers for spot-check proofs at trace positions; integrate into `try_compute_late_binding_proofs` and the verifier symmetric). Requires accessing the FRI commit’s merkle tree, exposed via `p3_commit::Pcs` or its backing MMCS.
2. **VEIL Lemma 4.7 implementation** (`crates/stark/src/whir_late_binding.rs`, new `commit_interleaved_codeword`, `open_with_post_commit_rho`, plus the spot-check protocol described in §10). The cleanest path uses “Path A” (commit g separately, spot check between g ’s codeword and the interleaved $\mathfrak{C}$ at K positions); avoids upstream patches.
3. **Recursion circuit verifier** (`crates/recursion/circuit/src/whir_verifier.rs`): consume the new opening shape, mirror the host `verify_chip_late_binding` logic in-circuit.
4. **gnark BN254 wrap artifact regen**: blocked on VK map regen (`vk not allowed` error in `build_plonk_bn254`); independent of soundness work.

References

- [1] U. Haböck, D. Levit, S. Papini. *WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification*. IACR ePrint 2024/1586, 2024.
- [2] R. Dalal, T. Hemo, E. Rabinovich, R. D. Rothblum. *VEIL: Lightweight Zero-Knowledge for Hash-Based Multilinear Proof Systems*. Succinct, March 2026.
- [3] A. Haböck. *Multivariate lookups based on logarithmic derivatives*. IACR ePrint 2022/1530, 2022.
- [4] C. Lund, L. Fortnow, H. Karloff, N. Nisan. *Algebraic methods for interactive proof systems*. J. ACM 39(4), 1992.
- [5] A. Golovnev, J. Lee, S. Setty, J. Thaler, R. S. Wahby. *Brakedown: Linear-time and field-agnostic SNARKs for R1CS*. CRYPTO 2023.
- [6] T. Hemo et al. *Jagged Polynomial Commitments*. IACR ePrint 2025/917, 2025.
- [7] The Plonky3 contributors. *Plonky3: A toolkit for STARK and SNARK proof systems*. <https://github.com/Plonky3/Plonky3>, accessed 2026.

- [8] ProjectZKM. *Ziren: A MIPS-based zero-knowledge virtual machine*. <https://github.com/ProjectZKM/Ziren>, accessed 2026.
- [9] Succinct Labs. *SP1: A high-performance zkVM with hypercube architecture*. <https://github.com/succinctlabs/sp1>, accessed 2026.